Sebenta 05

Orquestração de Containers com Kubernetes

Infraestruturas de Computação na Nuvem | Aula 05

Objeto e utilização deste documento

O presente documento acompanha a apresentação da Aula 05. Cada página reproduz um slide e desenvolve, em texto corrido, a matéria nele condensada. A aula é expositiva e assenta nos conceitos da Aula 04, em particular os namespaces, os cgroups e as imagens de container, que aqui reaparecem ao nível do cluster.

A leitura sequencial é a mais proveitosa. A aula parte da necessidade de orquestração, descreve a arquitetura do Kubernetes e os seus objetos, explica o princípio da reconciliação que lhe dá capacidade de recuperação automática, e percorre depois a rede, a configuração, o armazenamento, as cargas de trabalho pontuais e o agendamento. Termina com as alternativas ao Kubernetes.

Orquestração de Containers com Kubernetes

Infraestruturas de Computação na Nuvem, Aula 05

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

A Aula 04 mostrou que um container é um processo isolado por namespaces e limitado por cgroups, empacotado numa imagem e executado por um *runtime*. Tudo isso se passa num único anfitrião. Em produção, porém, uma aplicação raramente vive numa só máquina, e as questões que então surgem não são já de isolamento, mas de coordenação: onde correr cada container, quantas cópias manter, o que fazer quando uma máquina falha e como atualizar a aplicação sem a interromper.

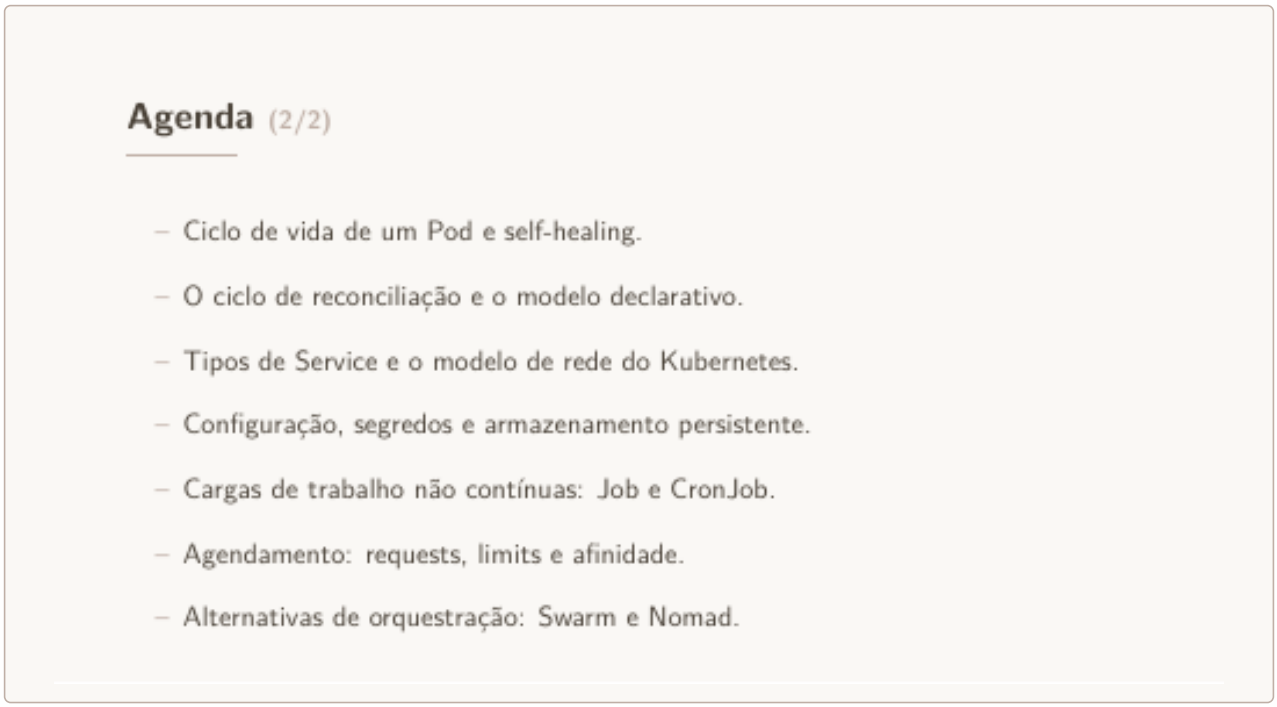
O Kubernetes é a resposta que a indústria consolidou para estas questões. Esta aula apresenta o seu modelo de funcionamento, mais do que os seus comandos. O objetivo é que, no final, o estudante saiba explicar o que acontece, passo a passo, desde que descreve uma aplicação num ficheiro até que ela está a correr, acessível e capaz de se recuperar de falhas.

Agenda (1/2)

- Porque precisamos de orquestração além de um único host.
- Arquitetura Kubernetes: control plane e worker nodes.
- Objetos principais: Pod, Deployment, ReplicaSet, Service, Namespace.

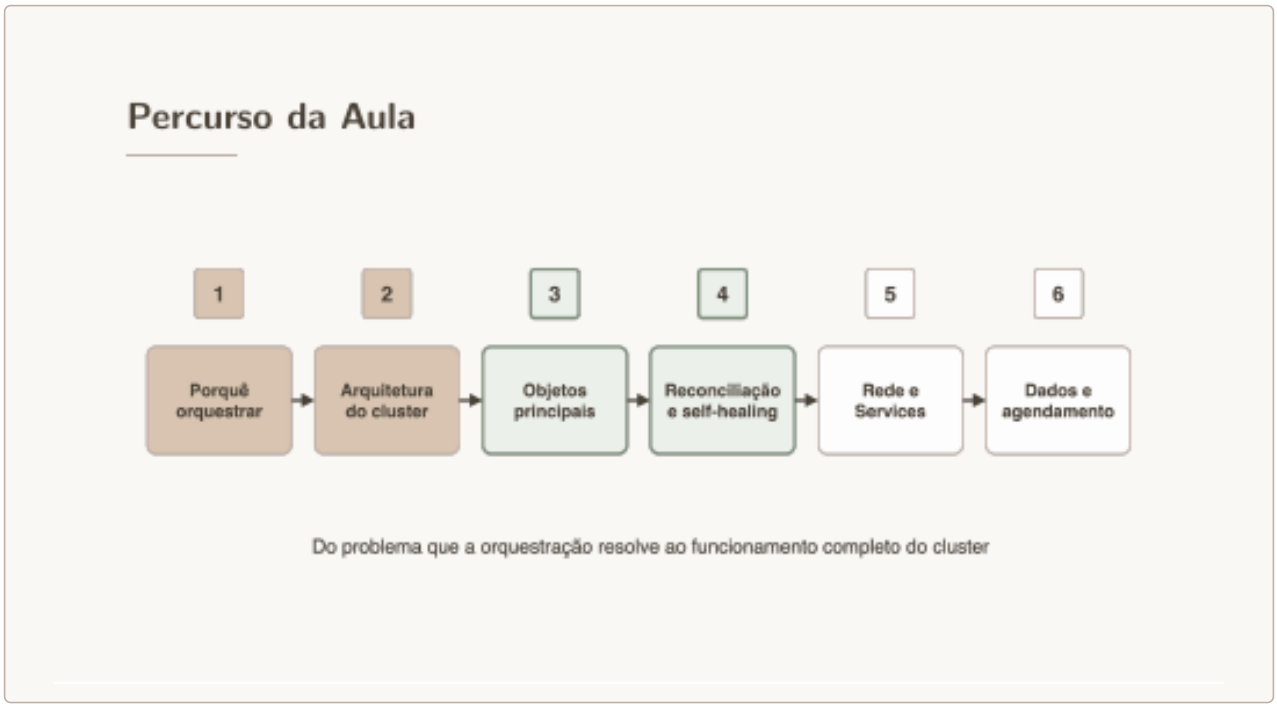
A primeira metade da agenda estabelece a motivação e a estrutura. Explica por que razão a gestão de containers num único anfitrião não basta, descreve a arquitetura do Kubernetes dividida entre o plano de controlo e os nós de trabalho, e apresenta os objetos com que se descreve uma aplicação: o Pod, o Deployment, o ReplicaSet, o Service e o Namespace.

Estes cinco objetos são o vocabulário mínimo. Quem os distinguir com clareza, e souber por que razão existe cada um, compreende a maior parte dos manifestos que encontrará em contexto profissional.



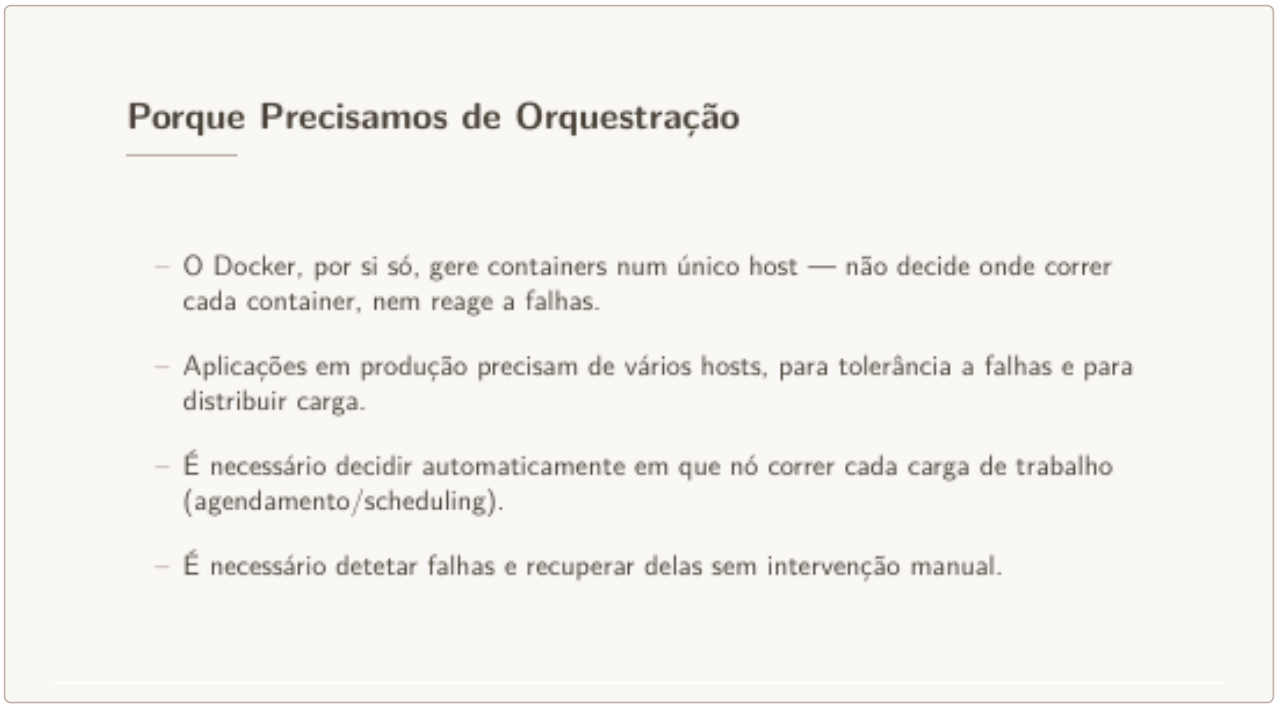
A segunda metade da agenda trata do comportamento do sistema e dos recursos que o rodeiam. Começa pelo ciclo de vida de um Pod e pela recuperação automática, e passa ao princípio que a explica, o ciclo de reconciliação do modelo declarativo. Segue-se a rede, com os tipos de Service e o modelo que garante a comunicação entre Pods, a configuração e os segredos, o armazenamento persistente e as cargas de trabalho que terminam, como os Jobs e os CronJobs.

A aula fecha com o agendamento, onde as noções de pedidos e limites ligam diretamente aos cgroups da Aula 04, e com uma comparação breve com o Docker Swarm e o Nomad, que ajuda a situar o Kubernetes no panorama dos orquestradores.



O percurso da aula organiza-se em seis blocos. O primeiro justifica a orquestração. O segundo descreve a arquitetura do cluster. O terceiro apresenta os objetos com que se descreve uma aplicação, e o quarto o princípio da reconciliação, de que decorre a recuperação automática. O quinto trata da rede e dos Services, e o sexto dos dados, da configuração e do agendamento.

As cores distinguem os blocos de base, a bege, os centrais, a verde, e os que completam o quadro, a branco. Quem tiver presente esta sequência consegue situar cada tema e perceber porque surge quando surge.

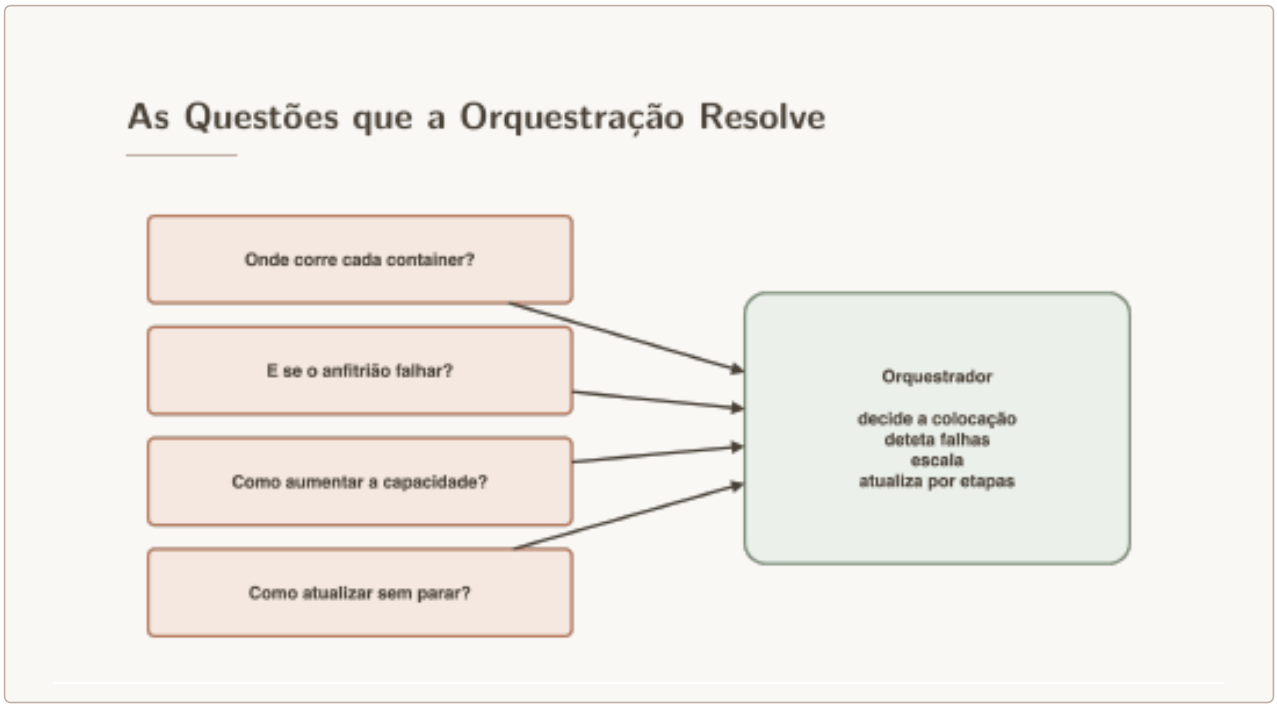


O Docker, por si só, gere containers num único anfitrião. Sabe criá-los, pará-los e ligá-los, mas não sabe escolher entre várias máquinas aquela onde cada um deve correr, e não reage quando uma máquina deixa de responder. Quem tenha o seu sistema a funcionar em três servidores teria de decidir manualmente onde colocar cada container e de intervir de cada vez que algo falhasse.

Em produção, as exigências são outras. A tolerância a falhas obriga a que a aplicação exista em várias máquinas, de modo que a perda de uma não a interrompa. A distribuição de carga obriga a repartir os pedidos por várias instâncias. A escala obriga a acrescentar ou retirar instâncias consoante a procura. As atualizações obrigam a substituir versões sem interromper o serviço.

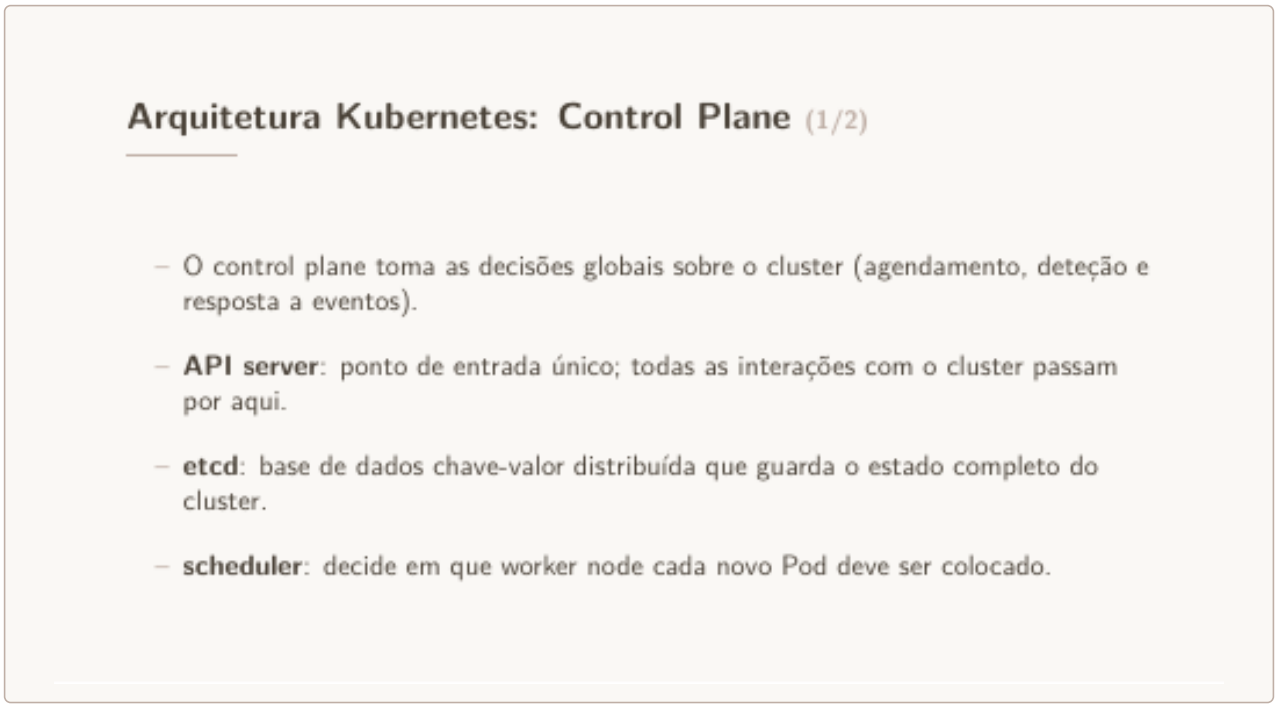
Orquestrar significa automatizar estas decisões. O orquestrador agenda cada carga de trabalho no nó adequado, deteta falhas e recupera delas sem intervenção humana, e mantém o sistema no estado que o administrador declarou. A ideia central, a de que se descreve o resultado pretendido e o sistema trabalha continuamente para o alcançar, é o fio condutor desta aula.

Quando estes problemas foram enfrentados à escala, os ensinamentos resultaram no Kubernetes, lançado pela Google em 2014 e inspirado no sistema interno Borg. Foi entregue à Cloud Native Computing Foundation e é hoje o orquestrador de referência.



As quatro perguntas à esquerda são aquelas a que o Docker, sozinho, não responde. A colocação consiste em escolher, entre várias máquinas, aquela onde cada container deve correr. A falha refere-se ao que acontece quando um anfitrião deixa de responder. A escala é o ajuste do número de instâncias à procura. A atualização é a troca de versões sem interrupção do serviço.

O orquestrador, à direita, assume estas decisões de forma automática e contínua. Não substitui o Docker nem o *runtime*. Acrescenta-lhes uma camada de decisão que opera sobre vários anfitriões.



Um cluster Kubernetes divide-se em duas partes. O plano de controlo, ou *control plane*, toma as decisões globais: onde colocar cada carga, como reagir a eventos, qual é o estado pretendido para cada recurso. Os nós de trabalho, ou *worker nodes*, executam efetivamente os containers.

O *API server* é o ponto de entrada único. Todas as interações com o cluster passam por ele, sejam as do administrador através do `kubectl`, as dos outros componentes ou as das próprias aplicações. Valida os pedidos, autentica e autoriza o requerente e persiste o resultado. Nenhum outro componente fala diretamente com a base de dados.

O *etcd* é essa base de dados, um repositório chave-valor distribuído que guarda o estado completo do cluster: que objetos existem, como foram declarados e em que estado se encontram. Usa um protocolo de consenso para se manter coerente entre várias réplicas, o que obriga a um número ímpar de membros, tipicamente três ou cinco. Perder o etcd sem cópia de segurança é perder o cluster.

O *scheduler* decide em que nó cada novo Pod deve correr. Primeiro filtra os nós que não satisfazem os requisitos, por falta de recursos ou por restrições declaradas, e depois classifica os restantes para escolher o mais adequado. Limita-se a decidir: quem lança o Pod é o nó escolhido.

Arquitetura Kubernetes: Control Plane (2/2)

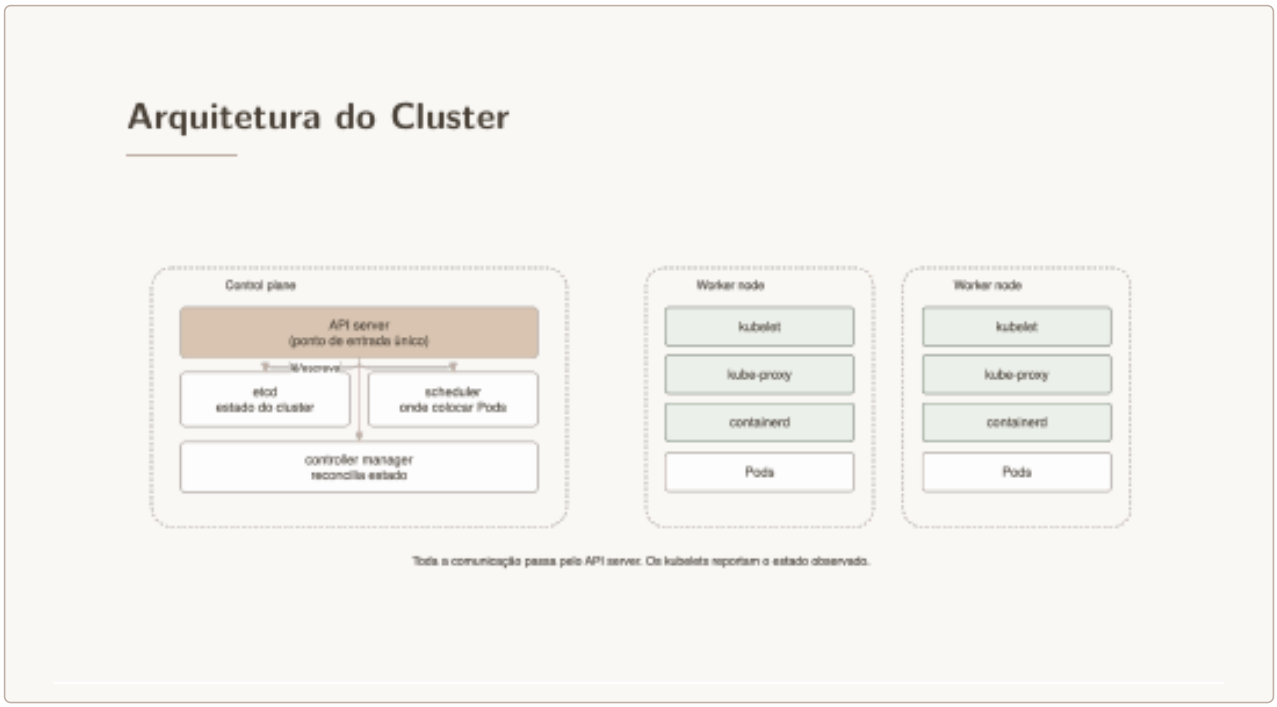
- **controller manager**: corre os controladores que comparam o estado atual com o estado desejado e agem para os aproximar.
- Os **worker nodes** são onde as cargas de trabalho efetivamente correm.
- **kubelet**: agente que garante que os containers descritos para o nó estão de facto a correr.
- **kube-proxy** e o **container runtime** (ex.: containerd) completam o nó, tratando de rede e execução de containers.

O *controller manager* executa os controladores, que são o motor do modelo declarativo. Cada controlador observa um tipo de recurso, compara o estado atual com o estado desejado e age para os aproximar. Existe um para os nós, um para os ReplicaSets, um para os Jobs e muitos outros, empacotados num único processo por conveniência. É deles que depende, no fim, a capacidade de recuperação do sistema.

Nos nós de trabalho, o *kubelet* é o agente que garante que os containers descritos para o nó estão a correr e saudáveis. Recebe do API server a lista de Pods que lhe foram atribuídos, instrui o *runtime* para os criar e comunica o estado observado. Sem o kubelet, o nó não participa no cluster.

O *kube-proxy* implementa em cada nó as regras de rede que fazem um Service encaminhar o tráfego para os Pods corretos. O *container runtime*, como o `containerd` estudado na Aula 04, é quem cria os containers. O kubelet comunica com ele por uma interface normalizada, a Container Runtime Interface, o que permite trocar de *runtime* sem alterar o Kubernetes. Foi por esta razão que o Kubernetes deixou de depender do Docker como intermediário e passou a usar diretamente o `containerd` ou o CRI-O.

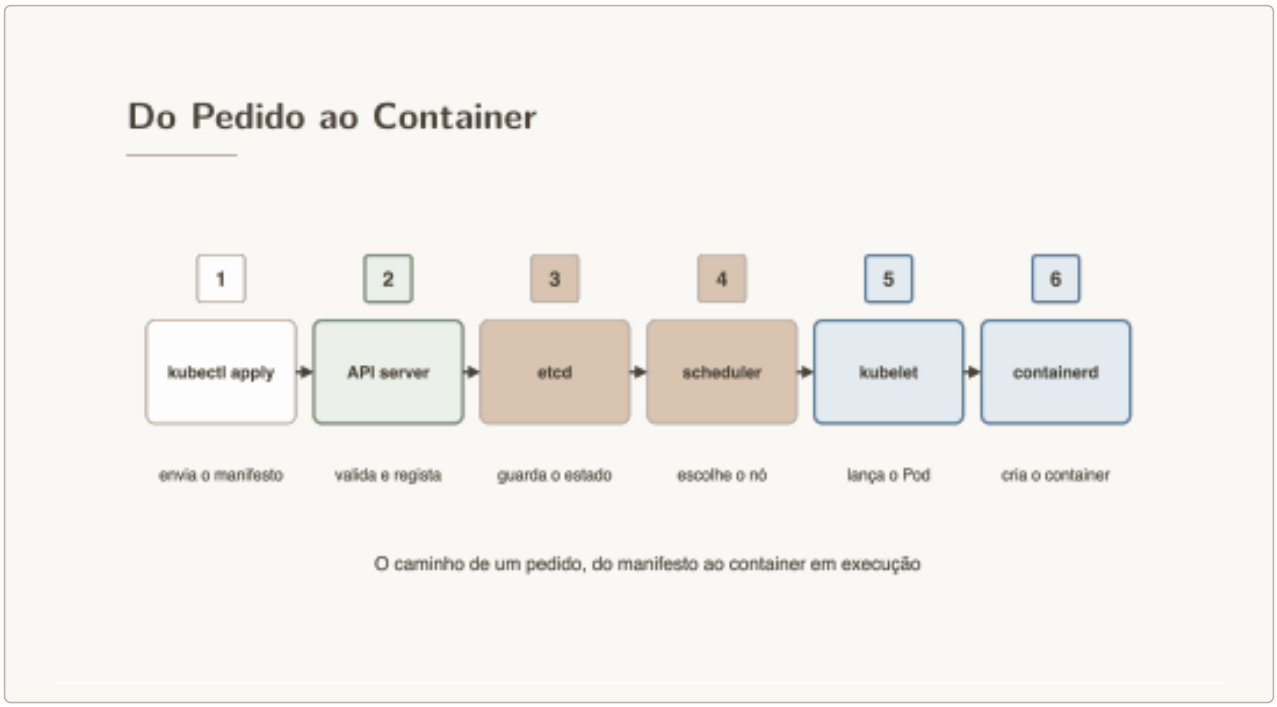
Dois pontos de leitura úteis. O plano de controlo pode, num cluster pequeno, coexistir no mesmo nó com cargas de trabalho, mas por omissão os nós de controlo recusam-nas através de uma marca que se estuda mais adiante. E todos os componentes comunicam por intermédio do API server, nunca diretamente entre si.



O diagrama mostra as duas partes do cluster e os componentes de cada uma. No plano de controlo estão o API server, o etcd, o scheduler e o controller manager. Em cada nó de trabalho estão o kubelet, o kube-proxy e o *runtime*, que é o `containerd`, e por cima deles os Pods.

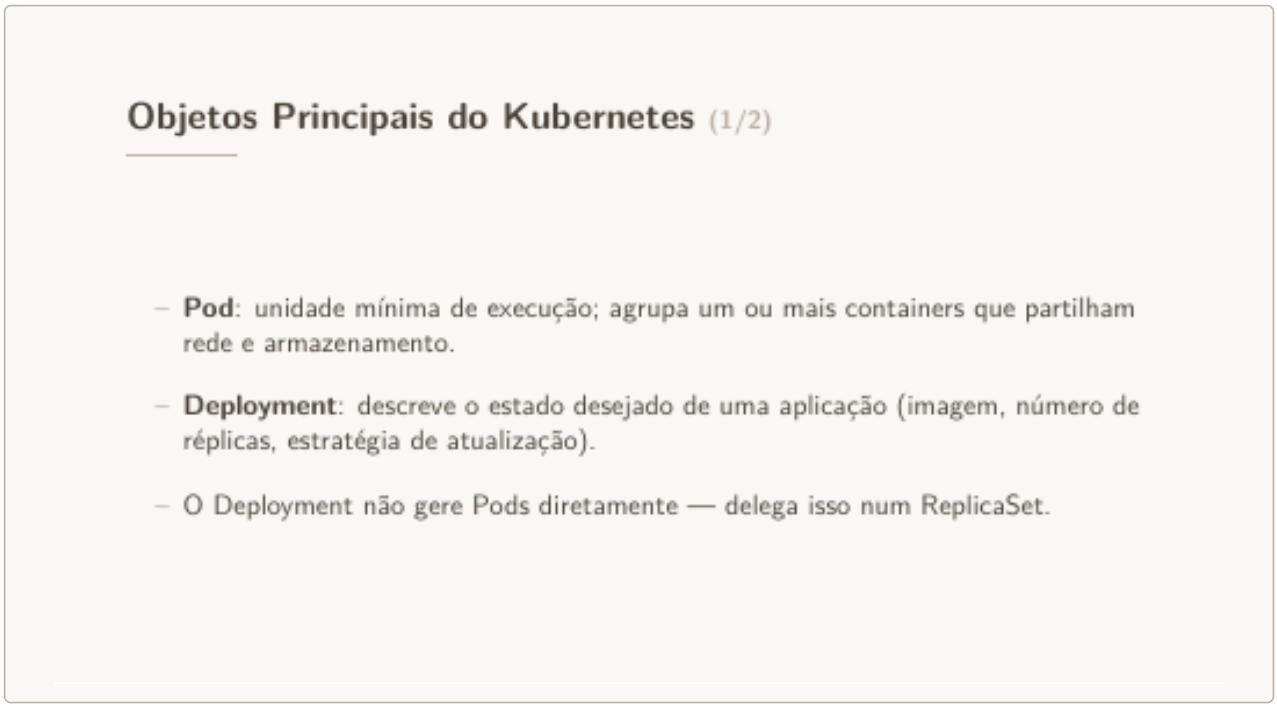
A legenda inferior resume o princípio de funcionamento. Toda a comunicação passa pelo API server, e os kubelets reportam o estado observado. Os outros componentes não se falam entre si: observam o API server e reagem ao que lá é registado. Esta arquitetura, centrada num ponto único de verdade, facilita o raciocínio sobre o sistema e é a razão pela qual a falha de um componente do plano de controlo não faz cair as cargas já em execução. Os nós continuam a executar os Pods que já têm, e apenas deixa de ser possível alterar o estado do cluster até o componente ser recuperado.

Sugere-se que o estudante reproduza este diagrama de memória, identificando para cada seta quem lê e quem escreve. É um bom teste da compreensão da arquitetura.



A figura segue um pedido de criação de uma aplicação. O `kubectl apply` envia o manifesto ao API server. Este valida o pedido, verifica as permissões e regista o objeto. O estado fica guardado no etcd, que é a única fonte de verdade do cluster. O scheduler repara num Pod ainda sem nó atribuído e escolhe o mais adequado. O kubelet desse nó observa a atribuição e pede ao *runtime*, o containerd, que crie o container.

Nenhum componente chama diretamente outro. Cada um observa o API server e reage ao que lá encontra. A decisão do scheduler é apenas registada, e quem a executa é o kubelet.



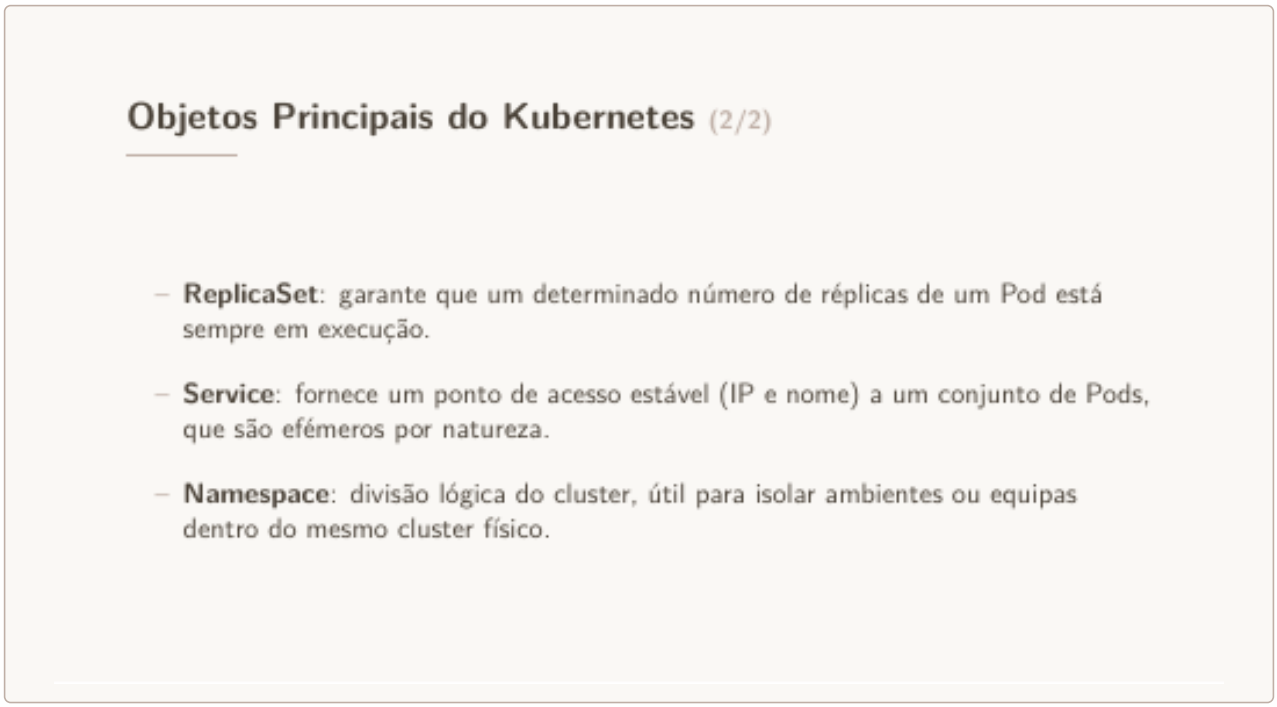
O Pod é a unidade mínima de execução do Kubernetes, e não o container. Um Pod agrupa um ou mais containers que partilham o mesmo NET namespace, e portanto o mesmo endereço IP e o mesmo espaço de portas, e que podem partilhar volumes. A analogia útil é a de uma pequena máquina lógica: os containers do Pod comunicam entre si por `localhost`, tal como processos da mesma máquina.

Na maioria dos casos, um Pod contém um único container. O agrupamento de vários justifica-se quando eles formam uma unidade inseparável, como uma aplicação e um pequeno auxiliar que lhe recolhe os registos ou lhe serve de proxy. Para aumentar a capacidade não se acrescentam containers a um Pod, criam-se mais Pods.

Um Pod é efémero por natureza. Recebe um endereço IP quando é criado e perde-o quando desaparece, e um Pod substituto recebe um endereço novo. Por essa razão, nunca se deve apontar para um Pod pelo seu endereço, e é este o problema que o Service resolve.

O Deployment descreve o estado desejado de uma aplicação: a imagem a usar, o número de réplicas e a estratégia de atualização. Em vez de criar Pods à mão, o administrador cria um Deployment e deixa que o sistema gira os Pods. Atualizar a aplicação é alterar a imagem no Deployment, e o Kubernetes substitui os Pods de forma gradual, por omissão, mantendo sempre capacidade disponível. Se a nova versão se comportar mal, é possível regressar à anterior.

O Deployment não gere os Pods diretamente. Delega essa tarefa num ReplicaSet, como se explica na página seguinte.

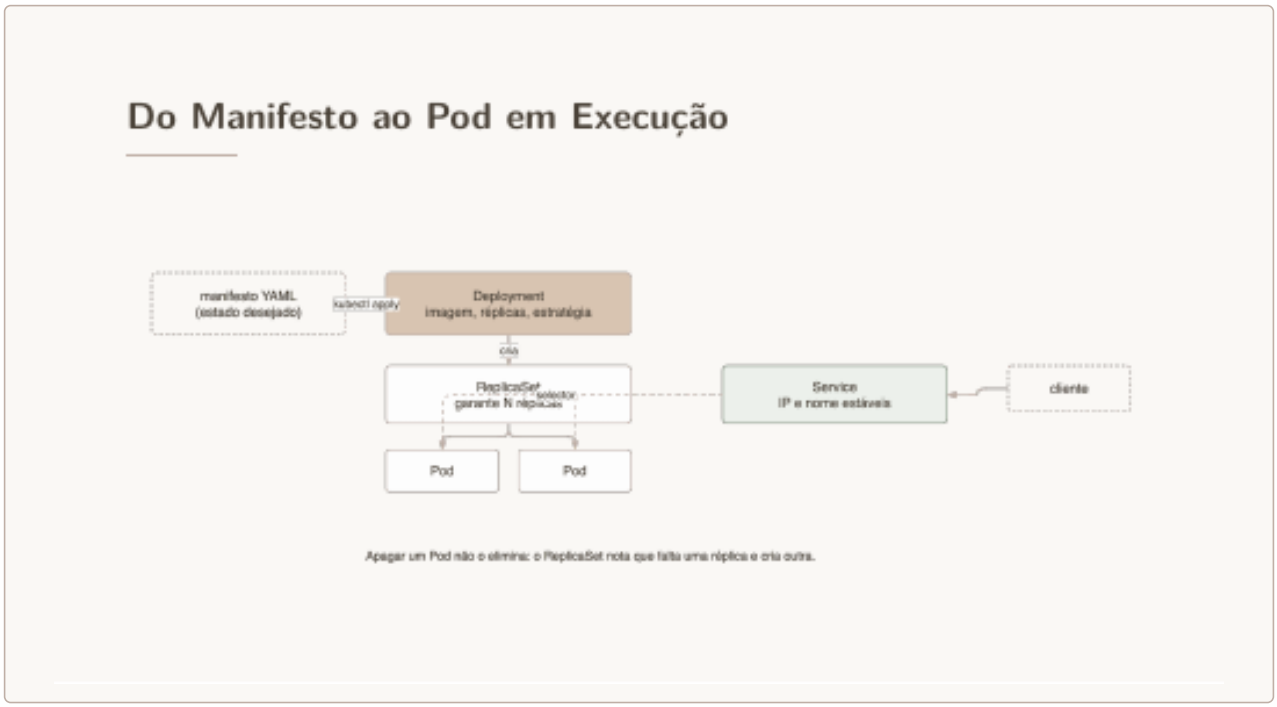


O ReplicaSet tem uma função única e precisa: garantir que um determinado número de réplicas de um Pod está sempre em execução. Se existirem menos, cria os que faltam. Se existirem mais, elimina os excedentes. Identifica os Pods que lhe pertencem através de etiquetas, e não por uma lista de nomes.

A separação entre Deployment e ReplicaSet parece redundante, mas serve a atualização. Quando se altera o Deployment, este cria um novo ReplicaSet para a versão nova e vai reduzindo o antigo à medida que o novo cresce. O ReplicaSet antigo é mantido, sem Pods, e é o que permite regressar rapidamente à versão anterior.

O Service fornece um ponto de acesso estável a um conjunto de Pods que, por serem efémeros, mudam de endereço. Tem um endereço IP virtual e um nome que se mantêm ao longo do tempo, e encaminha o tráfego para os Pods saudáveis que correspondem à sua etiqueta de seleção. O cliente fala com o Service e ignora que os Pods por trás dele nasçam e morram.

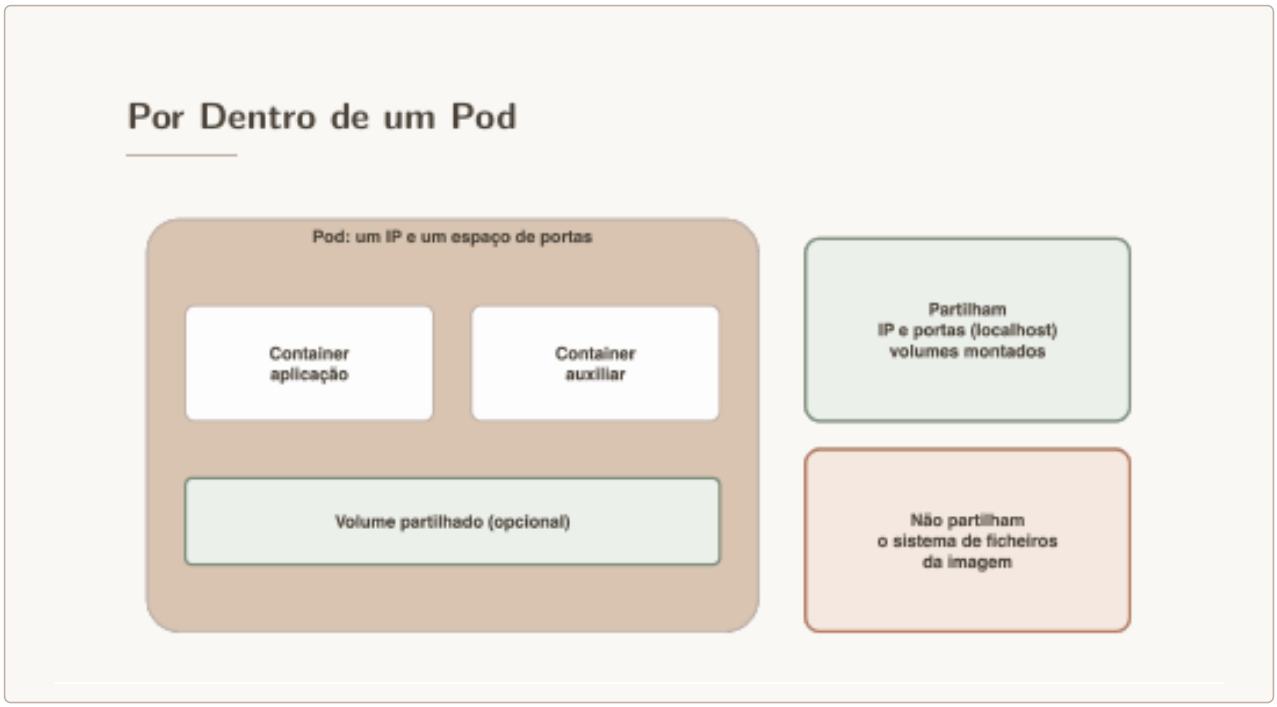
O Namespace do Kubernetes é uma divisão lógica do cluster, e não deve ser confundido com os namespaces do núcleo Linux da Aula 04, que são outra coisa. Serve para separar ambientes ou equipas dentro do mesmo cluster físico, permitindo que nomes se repitam em espaços diferentes e que se apliquem quotas e permissões por espaço.



O diagrama acompanha o percurso completo, do manifesto ao Pod. O administrador escreve um manifesto YAML com o estado desejado e entrega-o com `kubectl apply`. O Deployment cria o ReplicaSet, que cria os Pods. O Service, mais abaixo, escolhe os Pods por *selector*, e o cliente fala com o Service.

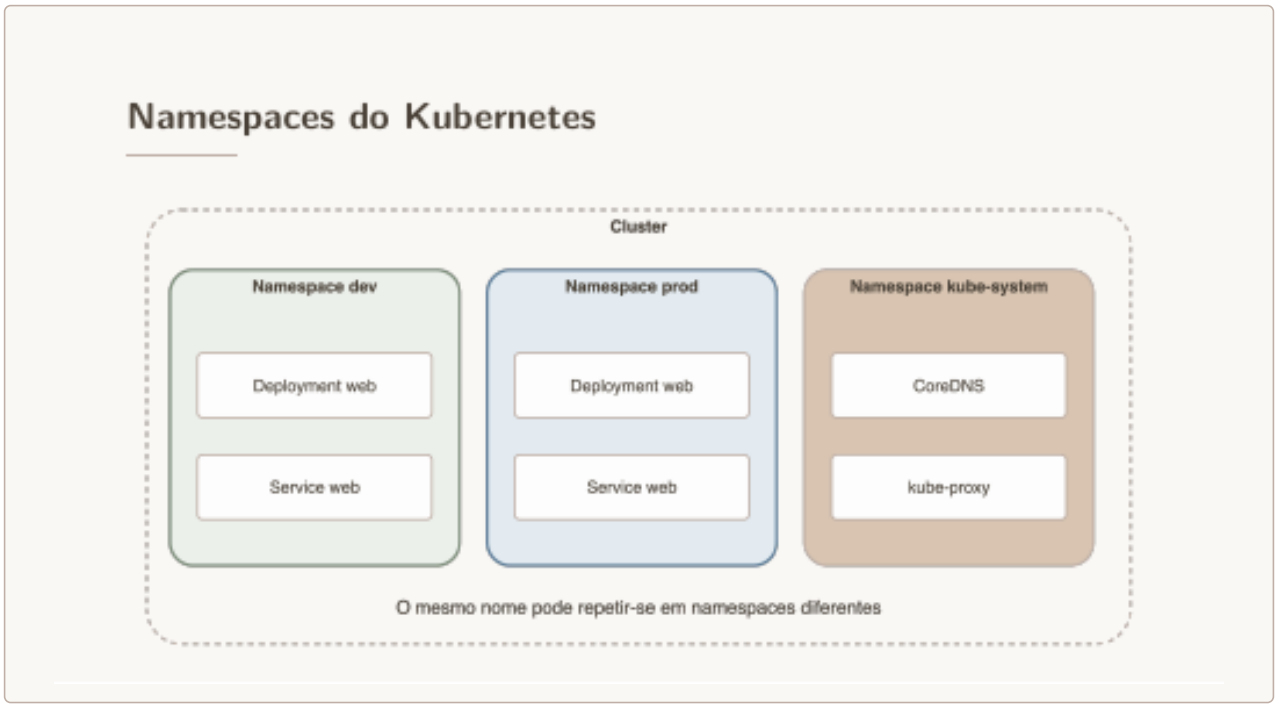
A legenda inferior condensa uma propriedade que parece surpreendente à primeira vista: apagar um Pod não o elimina. O ReplicaSet nota que falta uma réplica e cria outra. Quem queira realmente remover a aplicação tem de remover o objeto que a descreve, o Deployment, e não os Pods que dele resultam.

Repare-se também na ligação entre o Service e os Pods. Não é uma lista de endereços mantida à mão, mas uma regra de seleção por etiquetas. Quando nascem novos Pods com a etiqueta certa, o Service passa a incluí-los sem ser reconfigurado, e quando um desaparece deixa de lhe enviar tráfego. É esta indireção que permite que a aplicação cresça, encolha e se atualize sem que os clientes o percebam.



Um Pod agrupa containers que partilham contexto. O endereço IP e o espaço de portas são comuns, pelo que os containers comunicam entre si por `localhost` e não podem escutar na mesma porta. Os volumes declarados no Pod podem ser montados por vários containers, o que permite, por exemplo, que uma aplicação escreva registos que um auxiliar lê.

O que não se partilha é o sistema de ficheiros da imagem: cada container tem a sua raiz, construída a partir da sua imagem. Este arranjo decorre dos namespaces estudados na Aula 04, em particular do de rede, comum a todos os containers do Pod.



Um Namespace do Kubernetes é uma divisão lógica do cluster. Os objetos de cada espaço têm nomes próprios, pelo que o mesmo nome, como *web*, pode existir em *dev* e em *prod* sem conflito. Podem aplicar-se quotas de recursos e permissões por espaço, o que permite partilhar um cluster físico entre ambientes ou equipas. Os componentes do próprio sistema vivem em *kube-system*.

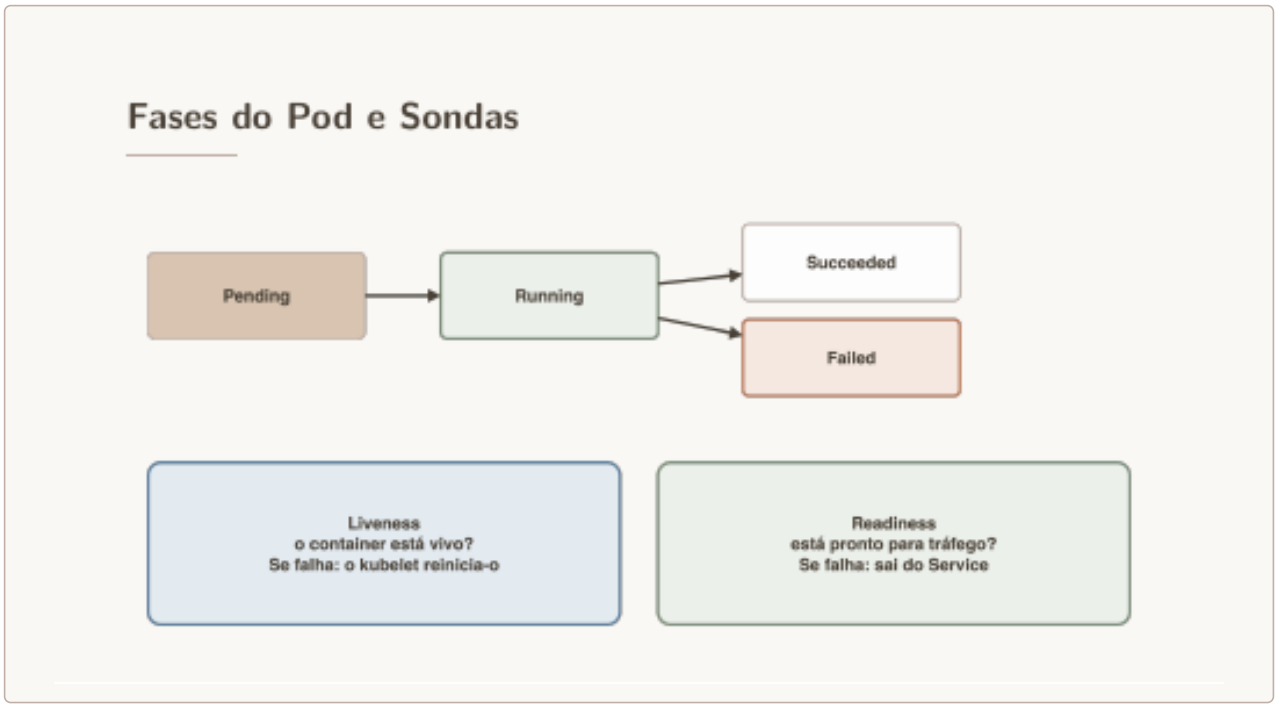
A designação coincide com a dos namespaces do núcleo Linux da Aula 04, mas os conceitos são distintos. Os do núcleo isolam processos. Os do Kubernetes organizam objetos.



Um Pod atravessa um conjunto de fases bem definidas. Fica em *Pending* enquanto aguarda colocação ou a transferência da imagem. Passa a *Running* quando pelo menos um container está em execução. Termina em *Succeeded*, se todos os containers terminaram com sucesso, ou em *Failed*, se algum terminou com erro. Existe ainda a fase *Unknown*, quando o estado não pode ser obtido, tipicamente por perda de contacto com o nó.

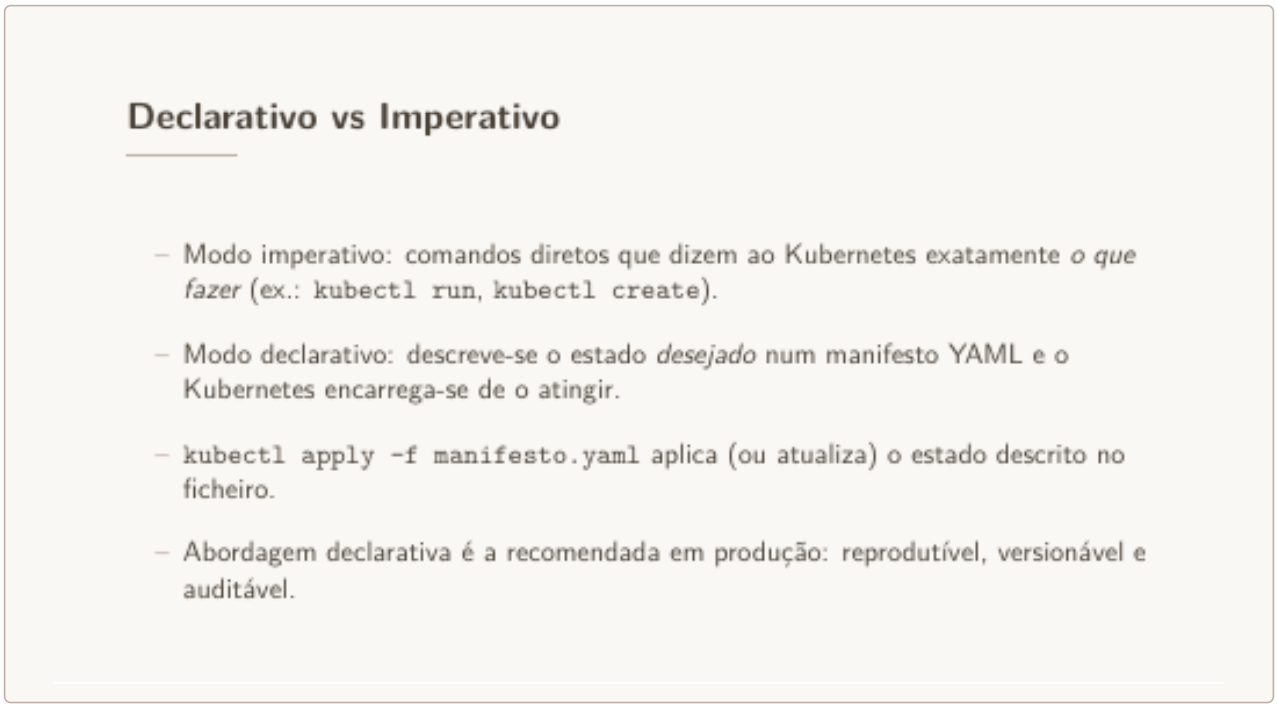
Estar em execução não quer dizer estar saudável. Para isso existem as sondas. A sonda de *liveness* verifica periodicamente se o container continua vivo, por exemplo através de um pedido HTTP, e se falhar, o kubelet reinicia-o. A sonda de *readiness* verifica se o container está preparado para receber tráfego, e enquanto falhar, o Pod é retirado do Service sem ser reiniciado. Uma aplicação que demora a arrancar, ou que perde temporariamente a ligação à base de dados, usa esta sonda para não receber pedidos que não consegue servir.

A recuperação automática atua a dois níveis. Se um container morre, o kubelet reinicia-o de acordo com a política do Pod. Se o Pod inteiro desaparece, ou o nó falha, o ReplicaSet cria um substituto noutro nó. O estado desejado, que o administrador declarou uma vez, é restaurado sem intervenção.



O Pod percorre fases bem definidas. Fica em *Pending* enquanto aguarda colocação ou a obtenção da imagem, passa a *Running* quando pelo menos um container está em execução e termina em *Succeeded* ou *Failed*. Estar em execução não implica estar saudável, e daí as sondas.

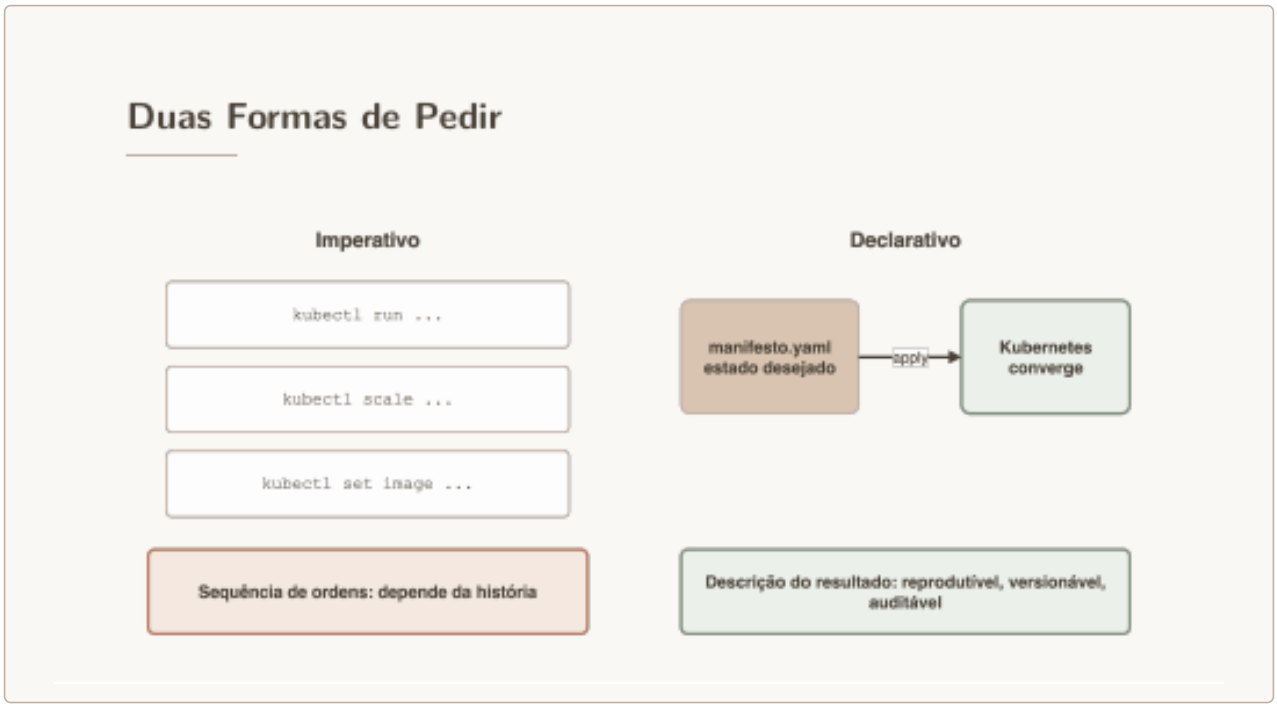
A de *liveness* responde à pergunta se o container está vivo, e a sua falha leva ao reinício pelo kubelet. A de *readiness* responde à pergunta se pode receber tráfego, e a sua falha retira o Pod do Service sem o reiniciar. São mecanismos distintos, com consequências distintas.



Existem duas formas de falar com o Kubernetes. No modo imperativo, o administrador emite comandos que dizem exatamente o que fazer, como criar um Pod ou escalar um Deployment. É rápido e adequado a experimentação, mas o resultado depende da sequência de comandos que foi executada e não deixa um registo reutilizável.

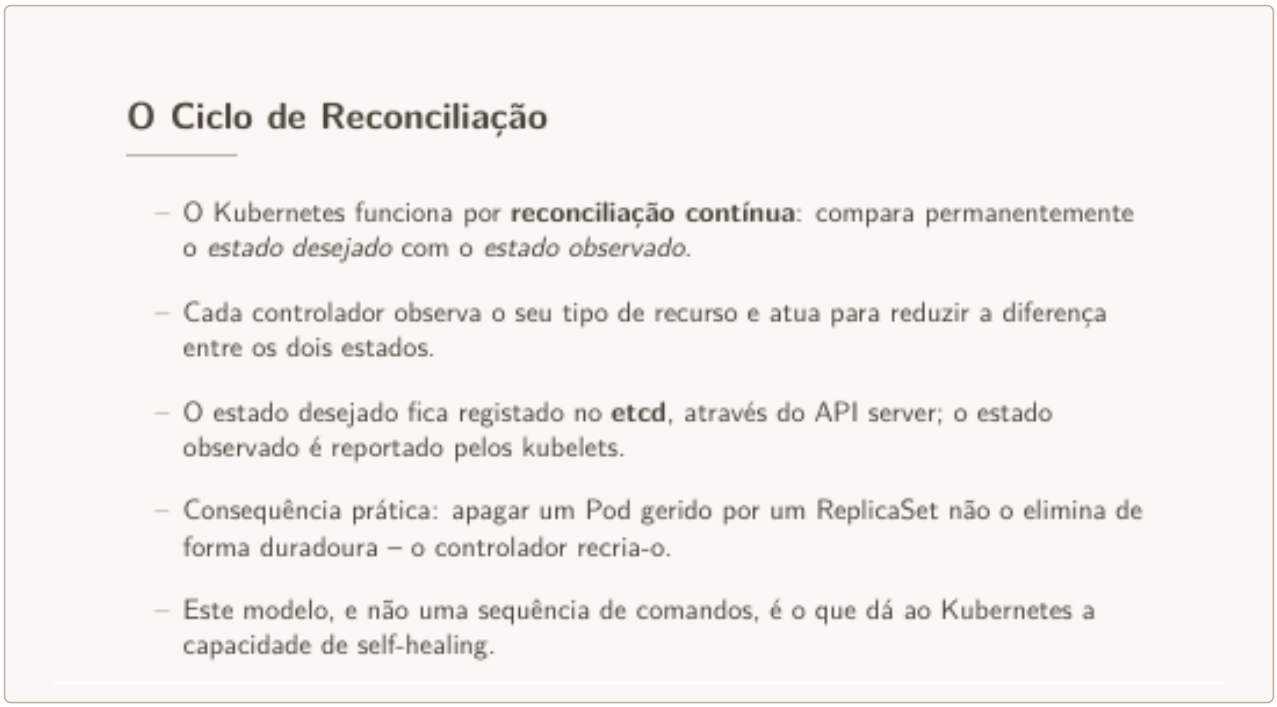
No modo declarativo, o administrador descreve num manifesto YAML o estado que pretende, e o Kubernetes encarrega-se de lá chegar. O comando `kubectl apply` compara o que está descrito no ficheiro com o que existe e aplica a diferença, criando, alterando ou removendo o necessário. Executá-lo duas vezes tem o mesmo efeito que executá-lo uma, propriedade a que se chama idempotência.

A abordagem declarativa é a recomendada em produção por três razões. É reprodutível, porque o mesmo manifesto cria o mesmo resultado em qualquer cluster. É versionável, porque o ficheiro pode viver num repositório com histórico e revisão. E é auditável, porque o estado em que o sistema deveria estar está escrito e pode ser comparado com o estado em que está. Esta forma de trabalhar, em que o repositório é a fonte de verdade da infraestrutura, liga-se diretamente à automatização de configurações que se estuda na Aula 07.



No modo imperativo, o resultado é a soma das ordens emitidas. Para o reproduzir é preciso conhecer a sequência exata, e nada garante que o estado atual corresponda a ela. No modo declarativo, o manifesto descreve o estado final e o Kubernetes calcula a diferença para o estado atual.

Aplicar duas vezes o mesmo manifesto não altera o resultado, propriedade a que se chama idempotência. Por ser um ficheiro de texto, o manifesto pode ser guardado num repositório, revisto e auditado. Por esta razão, a abordagem declarativa é a recomendada em produção.

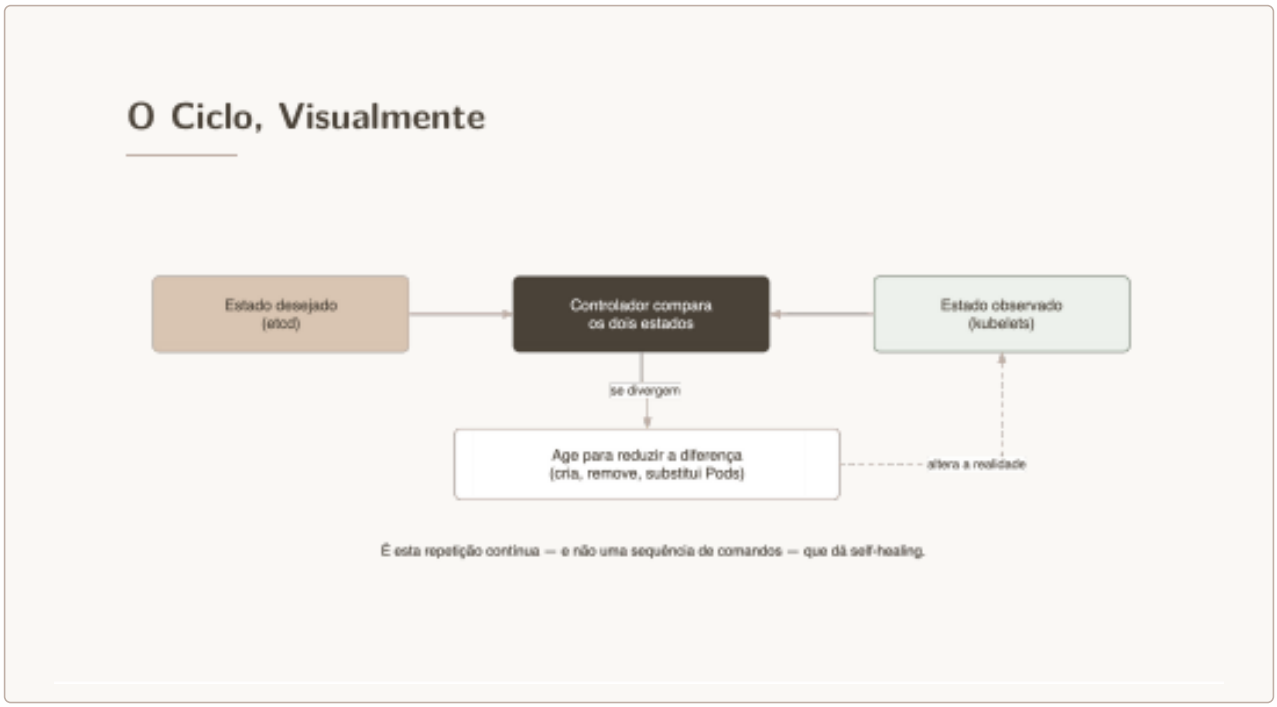


O princípio que sustenta tudo o que se viu é a reconciliação contínua. O Kubernetes não executa uma sequência de ordens e termina. Compara permanentemente o estado desejado, que o administrador declarou, com o estado observado, que os nós reportam, e age sempre que os dois divergem.

Cada controlador cuida de um tipo de recurso. Observa os objetos desse tipo através do API server, e quando deteta uma diferença, atua para a reduzir: cria Pods em falta, remove os excedentes, substitui os que falharam. O estado desejado fica registado no etcd por intermédio do API server, e o estado observado é comunicado pelos kubelets.

A consequência prática é a que o slide enuncia. Apagar um Pod gerido por um ReplicaSet não o elimina de forma duradoura, porque o controlador o recria. Não se trata de uma proteção especial contra erros, é apenas o comportamento natural de um sistema que persegue um estado.

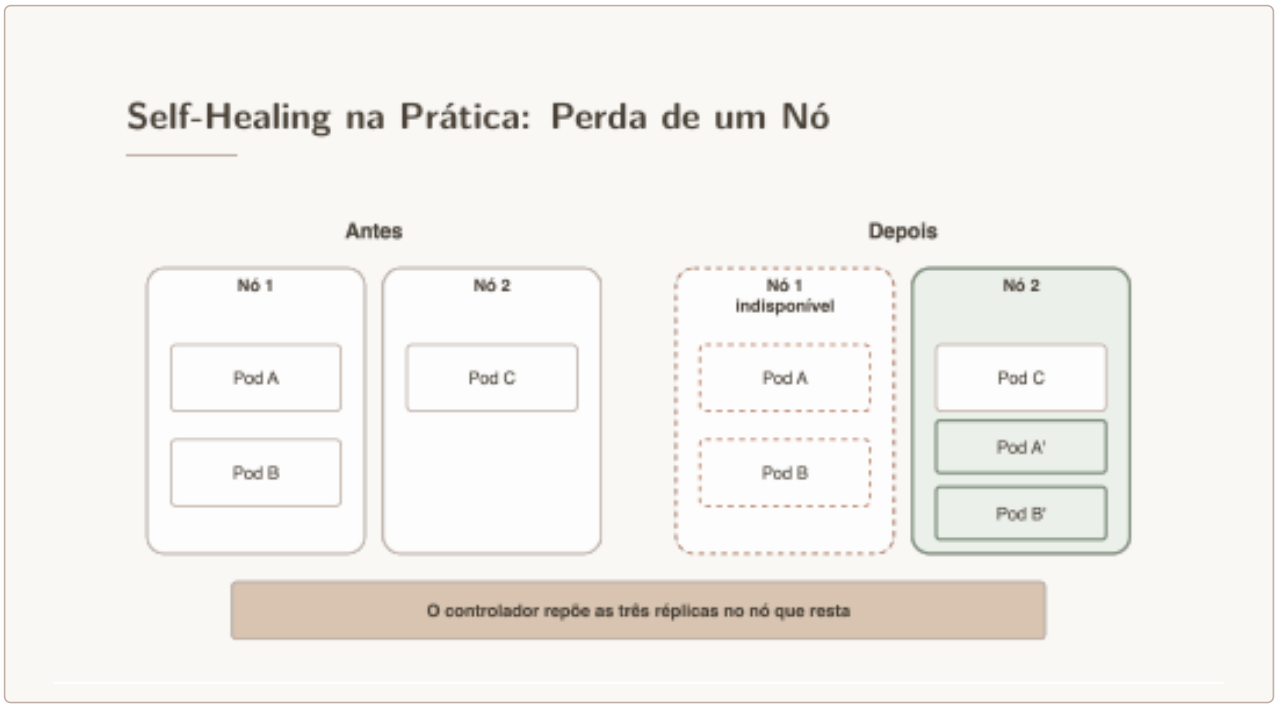
Este modelo é muito diferente do de um programa de comandos. Um *script* falha a meio e deixa o sistema num estado intermédio desconhecido. Um controlador, por voltar sempre a comparar, converge para o estado pretendido mesmo depois de falhas parciais. É esta propriedade, e não qualquer mecanismo isolado, que dá ao Kubernetes a capacidade de recuperação automática, o *self-healing*.



O diagrama representa o ciclo como um circuito. De um lado está o estado desejado, guardado no etcd. Do outro, o estado observado, reportado pelos kubelets. No meio, o controlador compara os dois e, se divergem, age para reduzir a diferença, o que altera a realidade, que por sua vez é de novo observada.

A legenda inferior frisa o ponto essencial: é a repetição contínua, e não uma sequência de comandos, que dá o *self-healing*. Cada passagem pelo ciclo é simples e pode falhar sem consequências graves, porque a seguinte corrige. A robustez vem da repetição.

Para consolidar, convém imaginar o ciclo em ação. Um nó perde-se. Os Pods que nele corriam deixam de ser reportados. O controlador do ReplicaSet verifica que o número de réplicas observadas é inferior ao desejado e cria Pods novos. O scheduler escolhe-lhes nós disponíveis. Os kubelets desses nós lançam os containers. Ninguém deu uma ordem, e o estado desejado foi restaurado. Quem consiga descrever esta cadeia sem consulta compreendeu a aula.



A figura aplica o ciclo de reconciliação a uma falha de nó. À esquerda, três Pods estão repartidos por dois nós. À direita, o nó 1 deixou de responder e os Pods que nele estavam deixam de ser contabilizados como disponíveis. O controlador do ReplicaSet verifica que existem menos réplicas do que as desejadas e cria substitutos, que o scheduler coloca no nó restante. Os Pods novos têm outros nomes e outros endereços.

A reação não é imediata. Por omissão, o Kubernetes tolera alguns minutos de silêncio de um nó antes de o dar como perdido, para não reagir a falhas breves.

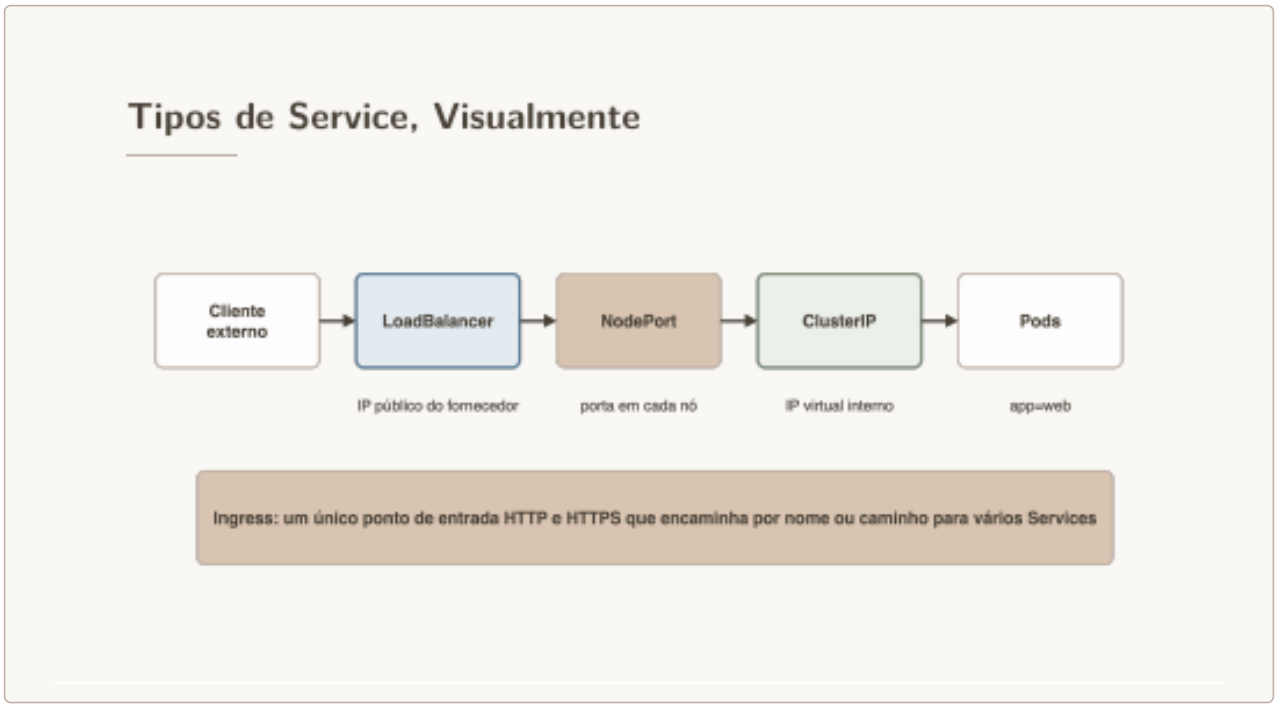


O Service é definido por um tipo, que determina quem pode alcançá-lo. O tipo *ClusterIP*, o de omissão, expõe o serviço apenas dentro do cluster, num endereço virtual interno. É o adequado à comunicação entre componentes da própria aplicação.

O tipo *NodePort* abre a mesma porta, tipicamente no intervalo de 30000 a 32767, em todos os nós do cluster, tornando o serviço acessível a partir do exterior através do endereço de qualquer nó. É simples, mas pouco elegante para produção. O tipo *LoadBalancer* pede um balanceador de carga externo ao fornecedor de nuvem, que lhe atribui um endereço público. Em clusters próprios, fora de um fornecedor, exige uma solução adicional que cumpra esse papel.

O *Ingress* não é um tipo de Service mas um recurso próprio. Encaminha tráfego HTTP e HTTPS por nome de máquina ou por caminho, o que permite que um único ponto de entrada sirva muitas aplicações. Precisa de um controlador de Ingress instalado no cluster para ter efeito, e a sua evolução é a Gateway API.

Em todos os casos, a escolha dos Pods de destino faz-se por *labels* e *selectors*. O Service declara o que procura, por exemplo todos os Pods com a etiqueta de uma certa aplicação, e o Kubernetes mantém atualizada a lista dos que satisfazem o critério e estão prontos. Nunca há endereços fixos.



Os tipos de Service constroem-se por camadas. O ClusterIP é a base: um endereço virtual interno, que dá acesso aos Pods dentro do cluster. O NodePort acrescenta uma porta aberta em todos os nós, normalmente entre 30000 e 32767, que encaminha para o ClusterIP. O LoadBalancer acrescenta um balanceador externo, pedido ao fornecedor de nuvem, que encaminha para as portas dos nós.

O Ingress é um recurso distinto. Recebe tráfego HTTP e HTTPS e distribui-o por vários Services segundo o nome da máquina ou o caminho. Em todos os casos, os Pods de destino são escolhidos por etiquetas.

O Modelo de Rede do Kubernetes

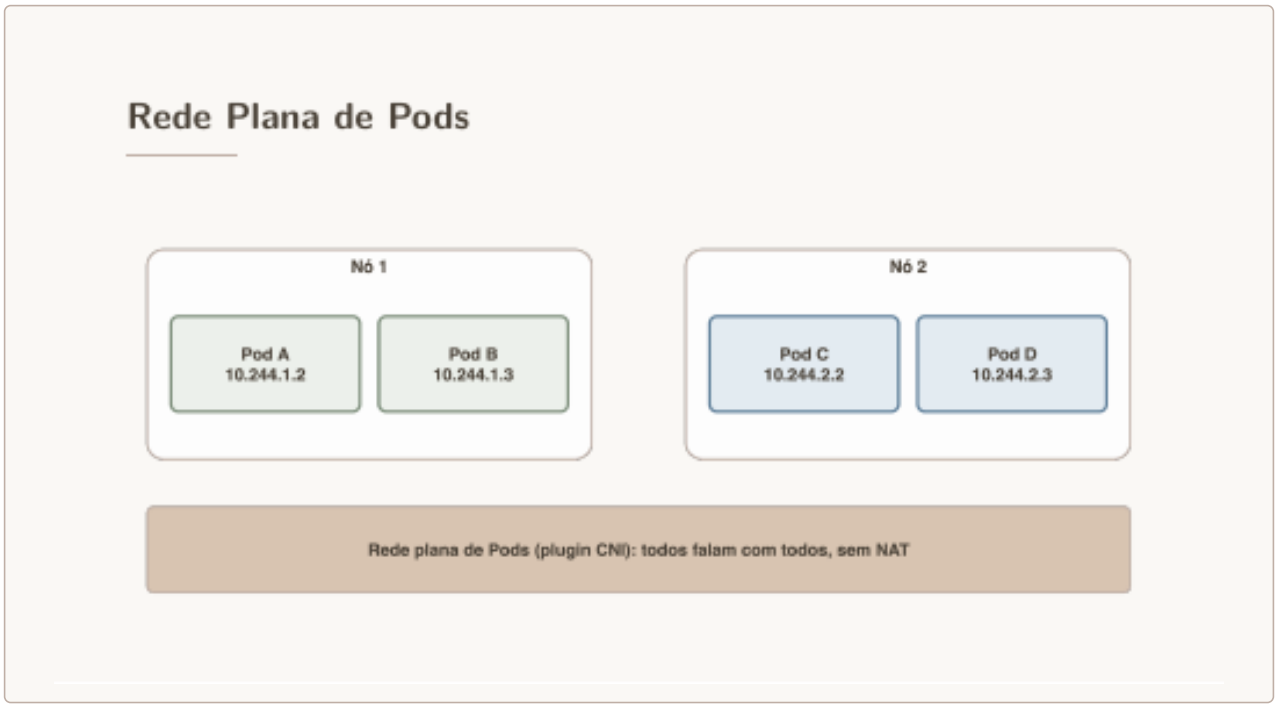
- Regra fundamental: cada Pod tem um endereço IP próprio e todos os Pods comunicam entre si sem NAT.
- Esta rede plana é implementada por um **plugin CNI** (Container Network Interface), não pelo Kubernetes em si.
- Exemplos de plugins CNI: Flannel, Calico, Weave Net, Cilium.
- **kube-proxy**: implementa em cada nó as regras que fazem um Service redirecionar tráfego para os Pods corretos.
- **CoreDNS**: fornece resolução de nomes interna, permitindo alcançar um Service pelo nome em vez do IP.
- Este modelo de rede será retomado na Aula 06, no contexto mais geral da SDN.

O modelo de rede do Kubernetes parte de uma regra simples e exigente. Cada Pod tem um endereço IP próprio e todos os Pods comunicam entre si diretamente, sem tradução de endereços, independentemente do nó em que estão. Visto de dentro, o cluster é uma rede plana.

O Kubernetes define esta regra, mas não a implementa. A implementação cabe a um *plugin* que cumpre a Container Network Interface. Existem vários, com abordagens diferentes: o Flannel, simples, que usa túneis entre nós; o Calico, que privilegia o encaminhamento e as políticas de rede; o Cilium, baseado em eBPF. O Weave Net, apresentado no slide, deixou de ter desenvolvimento ativo depois do encerramento da empresa que o mantinha, em 2024, pelo que é hoje mais relevante em clusters antigos do que em instalações novas.

O *kube-proxy* implementa em cada nó as regras que fazem um Service reencaminhar o tráfego para os Pods corretos. O modo tradicional usa regras de filtragem do núcleo, existindo alternativas por IPVS e, nas versões recentes, por nftables. O CoreDNS fornece a resolução de nomes interna: um Service é alcançado pelo nome, com a forma `servico.espaco.svc.cluster.local`, em vez de por um endereço.

Um aspeto prático merece ser retido. Sem um *plugin* de rede instalado, os nós ficam `NotReady` e o CoreDNS fica `Pending`, porque o nó não está preparado e o Pod não chega a ser colocado. É o sintoma habitual de uma instalação incompleta, e o gancho para a Aula 06, que aprofunda este modelo no contexto das redes definidas por software.



O modelo de rede do Kubernetes impõe uma regra simples: cada Pod tem o seu endereço IP e comunica com qualquer outro Pod sem tradução de endereços, estejam ou não no mesmo nó. Os endereços da figura, com o prefixo 10.244, são ilustrativos.

O Kubernetes não implementa esta rede. Delega-a num *plugin* que cumpre a Container Network Interface, como o Flannel, o Calico ou o Cilium. O kube-proxy trata dos Services e o CoreDNS dos nomes. Sem *plugin* de rede, os nós ficam *NotReady*, o que é o sintoma de uma instalação incompleta.

Configuração e Segredos

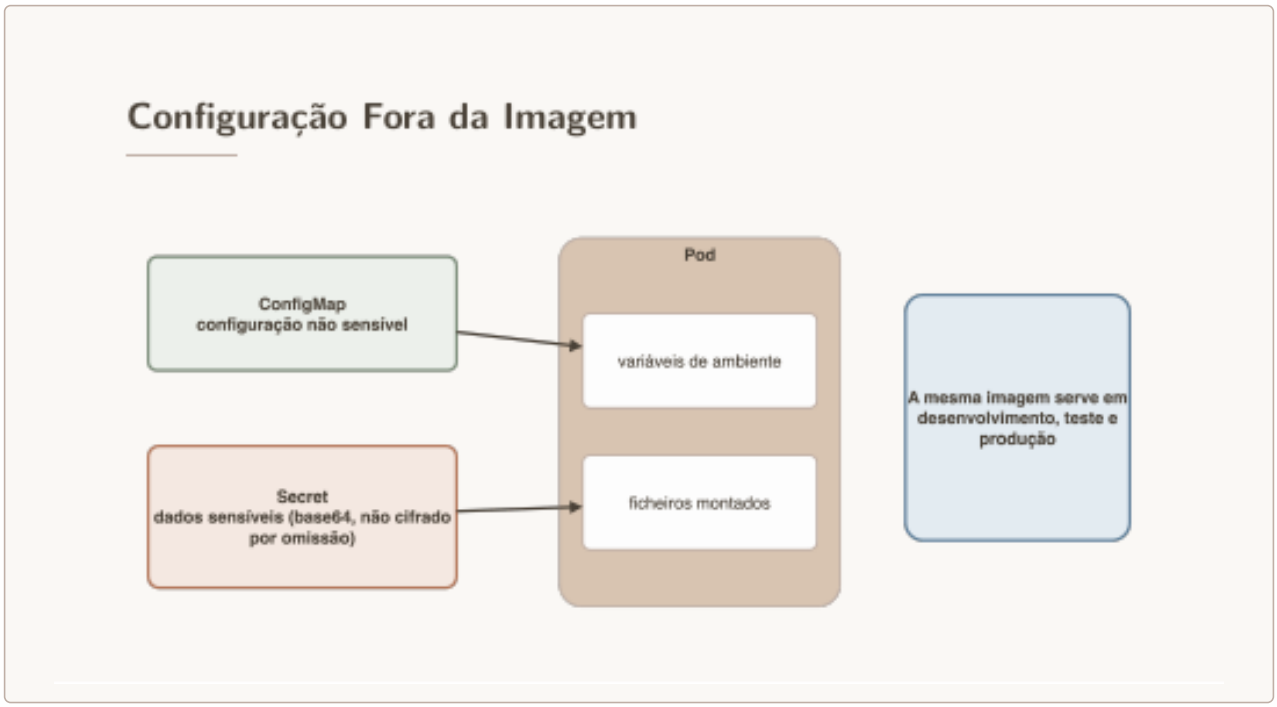
- **ConfigMap**: guarda dados de configuração não sensíveis, separando-os da imagem do container.
- **Secret**: destinado a dados sensíveis (passwords, chaves, tokens).
- Atenção: por omissão, os Secrets são apenas codificados em base64 – *não* cifrados. É preciso ativar cifra em repouso no etcd e restringir acessos por RBAC.
- Ambos podem ser expostos ao container como variáveis de ambiente ou como ficheiros montados.
- Separar configuração da imagem permite usar a mesma imagem em desenvolvimento, teste e produção.

Uma boa prática herdada da Aula 04 é manter a configuração fora da imagem. A mesma imagem deve servir em desenvolvimento, em teste e em produção, e o que muda entre ambientes, como endereços de bases de dados ou níveis de registo, é fornecido de fora. O Kubernetes tem dois objetos para isso.

O ConfigMap guarda dados de configuração não sensíveis, como pares chave-valor ou ficheiros inteiros. O Secret destina-se a dados sensíveis, como palavras-passe, chaves e *tokens*. Do ponto de vista de utilização são muito semelhantes: ambos podem ser expostos ao container como variáveis de ambiente ou como ficheiros montados numa pasta.

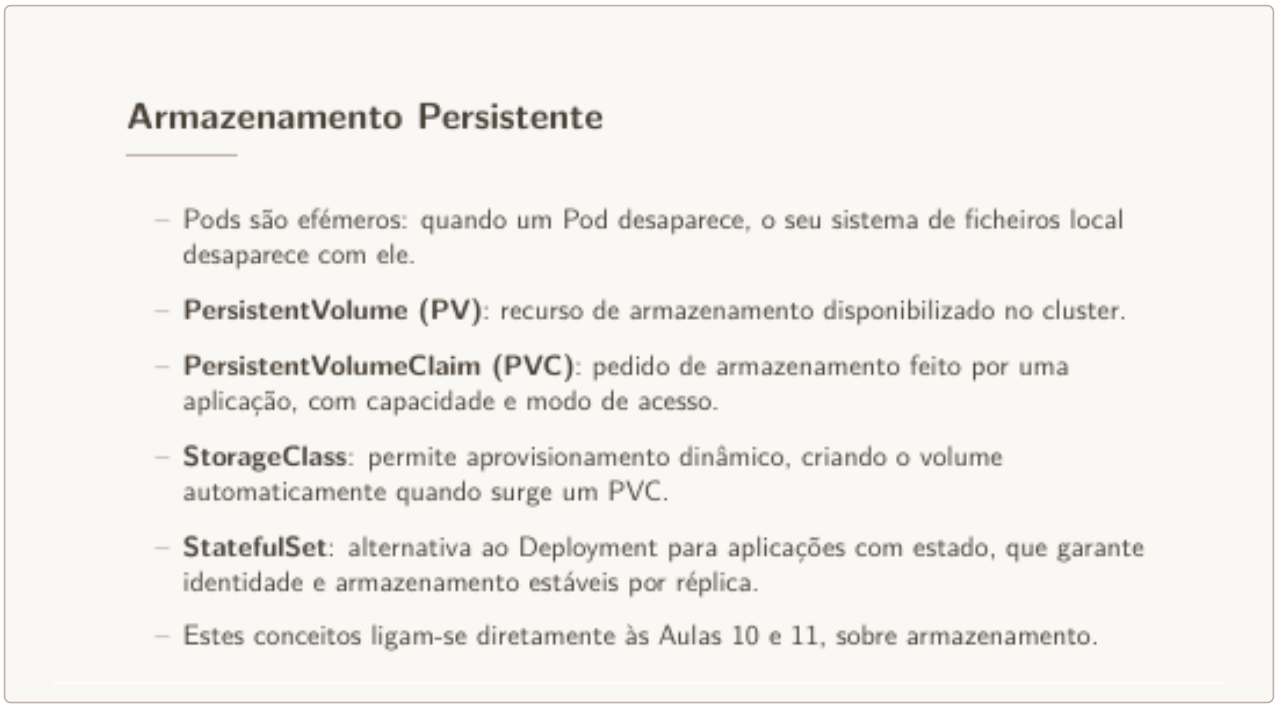
O aviso do slide é decisivo e frequentemente mal compreendido. Por omissão, os valores de um Secret estão apenas codificados em base64, o que é uma representação e não uma cifra, e qualquer pessoa que os leia os descodifica de imediato. Para um Secret ser realmente protegido é preciso ativar a cifra em repouso no etcd e restringir o acesso aos objetos com o controlo de acessos baseado em papéis, o RBAC. Quem tem permissão para ler Secrets num espaço de nomes lê as palavras-passe que lá estão.

Há uma diferença de comportamento a conhecer. Quando um ConfigMap é montado como ficheiros, as alterações acabam por ser propagadas ao container. Quando é injetado como variável de ambiente, o valor é fixado no arranque, e só um novo Pod vê a alteração.



A configuração é mantida fora da imagem para que a mesma imagem sirva em todos os ambientes. O ConfigMap guarda dados não sensíveis. O Secret destina-se a dados sensíveis, mas por omissão apenas os codifica em base64, o que não é cifra. A proteção real exige cifra em repouso no etcd e controlo de acessos por RBAC.

Ambos chegam ao container de duas formas. Como variáveis de ambiente, lidas no arranque, ou como ficheiros montados, que são atualizados quando o objeto muda. Daí que, no primeiro caso, uma alteração de configuração só chegue aos Pods quando estes são recriados.



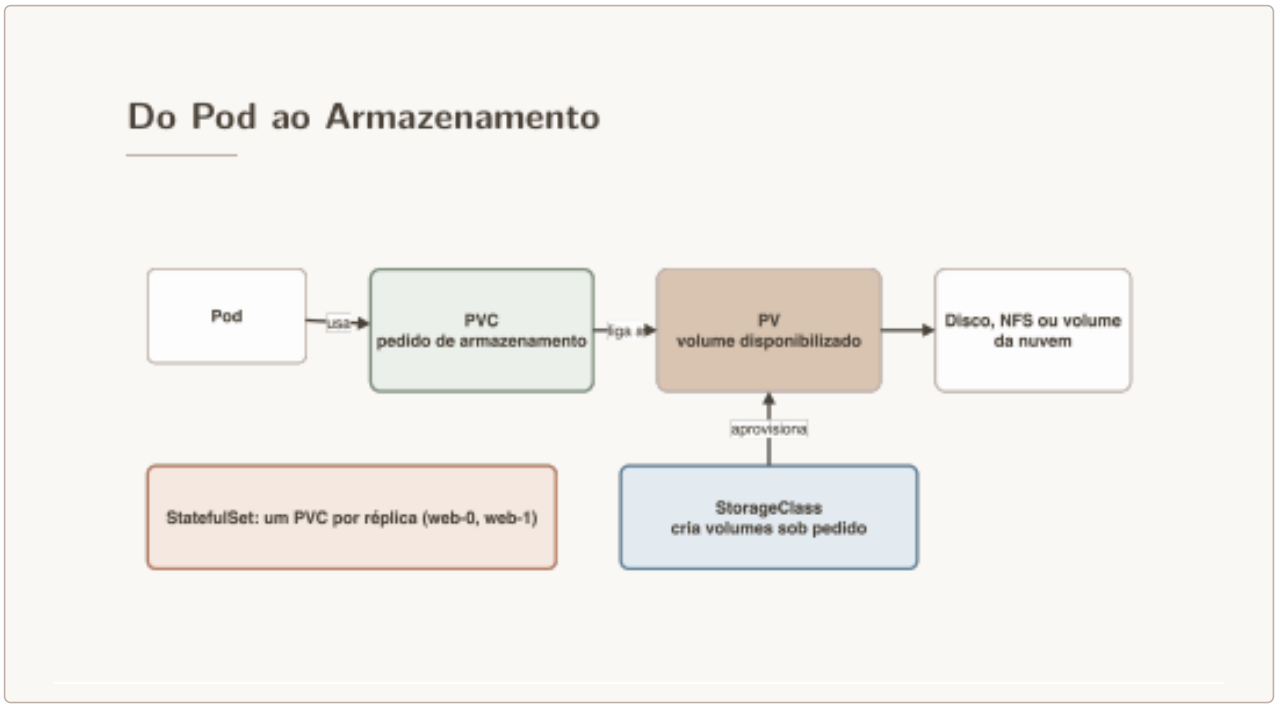
Os Pods são efémeros, e o seu sistema de ficheiros local desaparece com eles. É o mesmo princípio da camada de escrita do container, visto agora à escala do cluster. Para aplicações que precisam de guardar dados, como bases de dados, o Kubernetes separa o pedido de armazenamento do armazenamento em si.

Um PersistentVolume, ou PV, é um recurso de armazenamento disponibilizado no cluster, que pode corresponder a um disco de nuvem, a uma partilha de rede ou a um volume local. Um PersistentVolumeClaim, ou PVC, é o pedido feito por uma aplicação, com a capacidade pretendida e o modo de acesso, por exemplo leitura e escrita por um único nó ou por vários. O Kubernetes liga cada pedido a um volume que o satisfaça. A aplicação refere o pedido e ignora que tecnologia há por trás.

A StorageClass permite o aprovisionamento dinâmico. Em vez de um administrador criar volumes antecipadamente, a classe descreve como criá-los, e quando surge um pedido o volume é criado automaticamente. A classe define ainda o que acontece ao volume quando o pedido é removido: pode ser apagado ou retido.

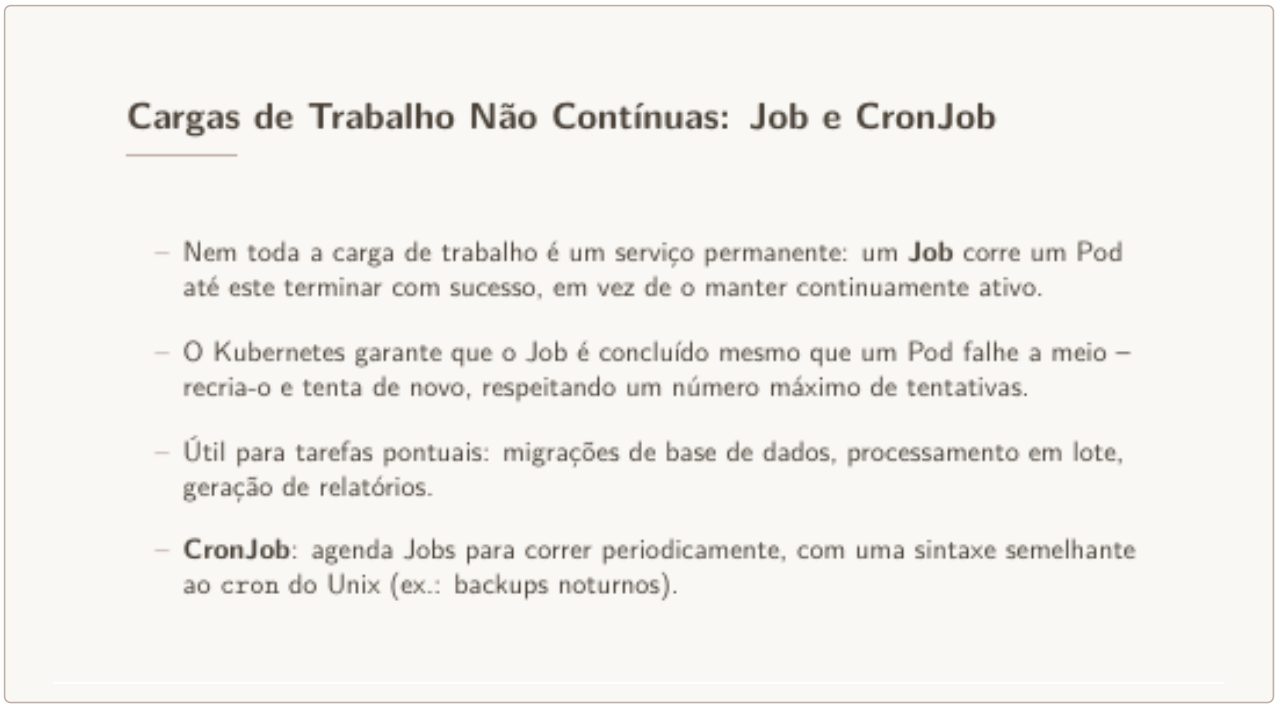
O StatefulSet é a alternativa ao Deployment para aplicações com estado. Atribui a cada réplica uma identidade estável, com nomes ordenados como `base-0` e `base-1`, e um volume próprio que acompanha a réplica ao longo das recriações. Numa base de dados replicada, a diferença entre réplicas não é opcional.

Estes conceitos retomam, com o vocabulário do Kubernetes, o que as Aulas 10 e 11 aprofundam sobre armazenamento local e distribuído.



O armazenamento persistente separa o pedido do recurso. A aplicação refere um PVC, que indica a capacidade e o modo de acesso pretendidos. O Kubernetes liga o pedido a um PV, que é o recurso disponibilizado e que corresponde a um disco, a uma partilha de rede ou a um volume da nuvem.

A StorageClass descreve como criar volumes de um dado tipo e permite o aprovisionamento dinâmico: o volume é criado quando surge o pedido. O StatefulSet atribui um PVC próprio a cada réplica, e esse volume acompanha a réplica quando esta é recriada.

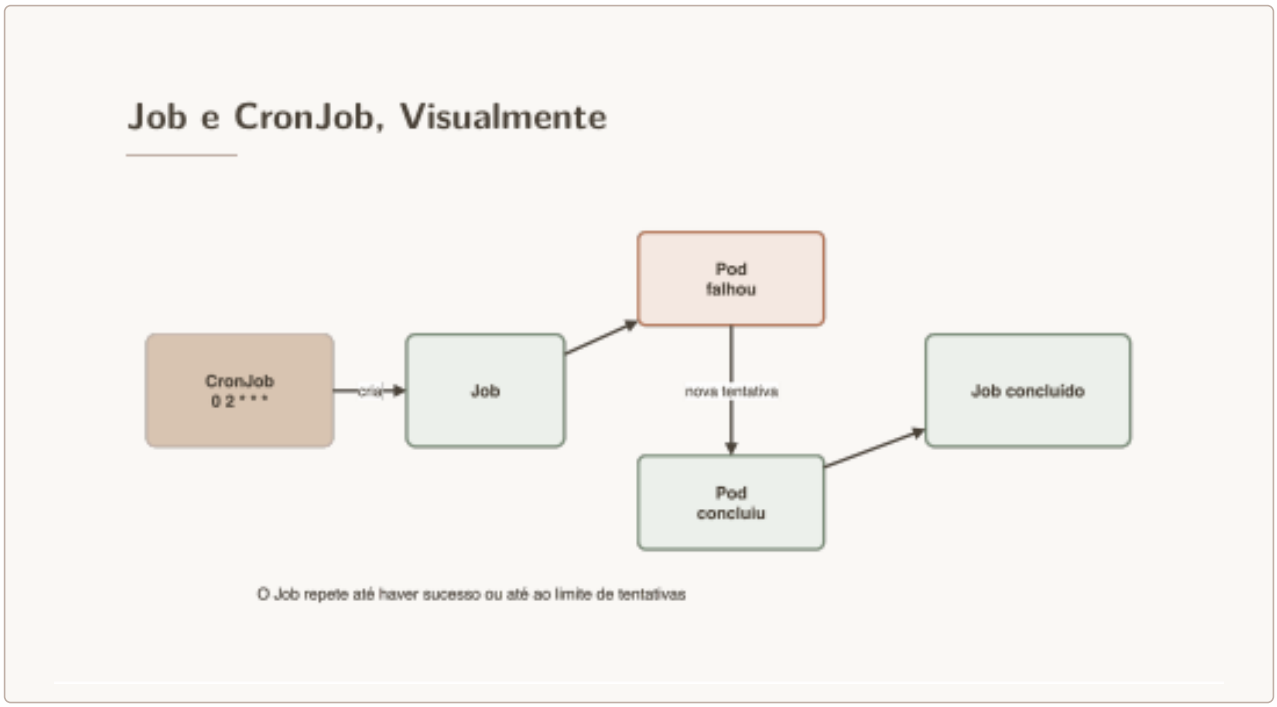


Nem toda a carga de trabalho é um serviço permanente. Há tarefas que têm princípio e fim, como migrar uma base de dados, processar um lote de ficheiros ou gerar um relatório. Para elas, o Deployment é inadequado, porque recriaria o Pod sempre que terminasse, e o Kubernetes oferece o Job.

Um Job corre um Pod até que este termine com sucesso, e dá a tarefa por concluída nesse momento. Se um Pod falhar a meio, cria outro e tenta de novo, até um número máximo de tentativas, após o qual o Job é dado como falhado. Pode ser configurado para exigir várias conclusões e para as executar em paralelo, o que o torna adequado a processamento em lote.

O CronJob agenda a criação de Jobs de forma periódica, com uma sintaxe de cinco campos semelhante à do `cron` do Unix. É o instrumento natural para tarefas como cópias de segurança noturnas ou limpezas regulares. Há decisões de configuração relevantes, como saber o que fazer se a execução anterior ainda não terminou, e que o agendamento é interpretado, por omissão, no fuso horário do plano de controlo.

Esta classificação das cargas em serviços contínuos, tarefas que terminam e tarefas periódicas é útil para além do Kubernetes. Condiciona a escolha do objeto, a política de reinício e a forma de monitorizar o resultado.



O Job executa um Pod até que este termine com sucesso. Se o Pod falhar, o Job cria outro e tenta de novo, até ao limite de tentativas configurado, a partir do qual é dado como falhado. O CronJob está uma camada acima: cria Jobs de acordo com uma expressão de agendamento de cinco campos, semelhante à do `cron`.

A figura mostra um agendamento às 2 horas, um Pod que falha, uma nova tentativa que conclui e o Job terminado. A classificação das cargas em serviços contínuos, tarefas com fim e tarefas periódicas condiciona a escolha do objeto.

Agendamento: Requests e Limits

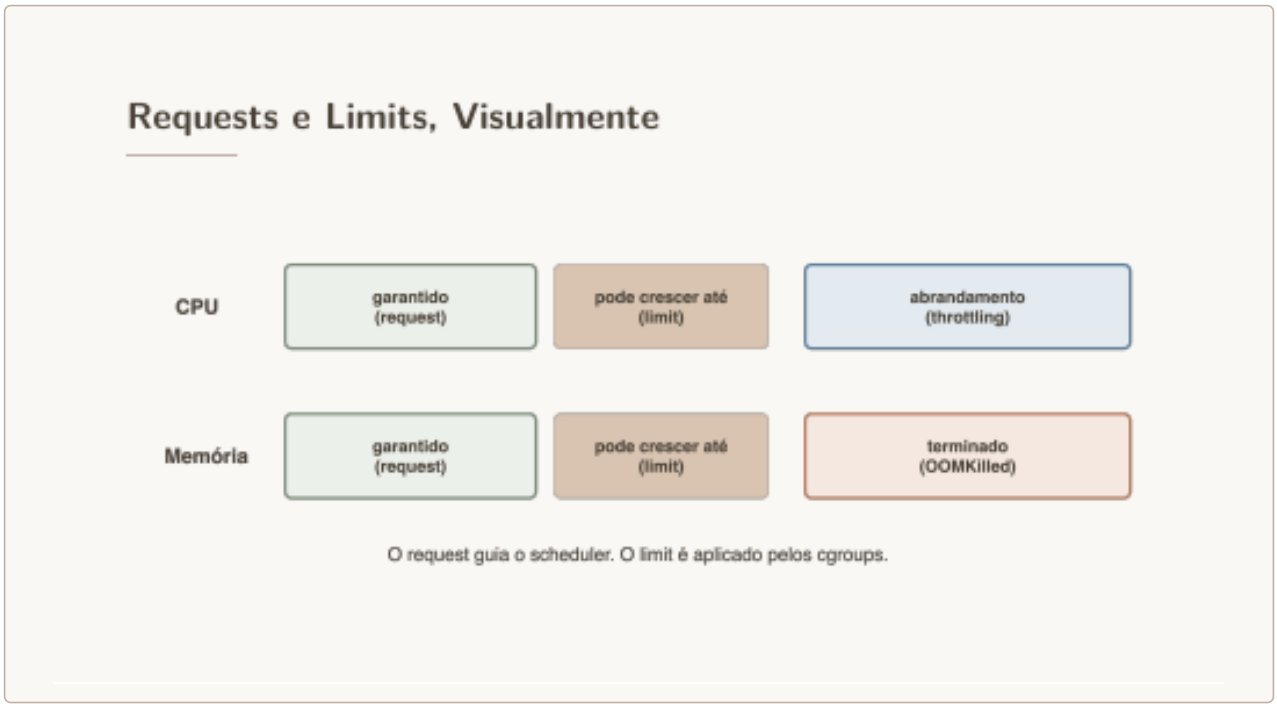
- **Requests**: recursos que o Pod garantidamente precisa; o scheduler usa este valor para escolher um nó com capacidade suficiente.
- **Limits**: teto máximo que o container pode consumir, aplicado através dos cgroups vistos na Aula 04.
- Exceder o limite de memória leva à terminação do container (OOM kill); exceder o de CPU resulta apenas em *throttling*.
- **nodeSelector** e **affinity**: influenciam em que nós um Pod pode ou deve ser colocado.
- **Taints e tolerations**: mecanismo inverso, em que um nó repele Pods que não tolerem explicitamente a sua marca.

O scheduler precisa de saber quanto cada Pod consome para o colocar bem. Cada container pode declarar *requests* e *limits*. O *request* é a quantidade de recursos que o container garantidamente precisa, e é o valor que o scheduler usa para escolher um nó com capacidade suficiente. O *limit* é o teto que o container não pode exceder, e é aplicado, no nó, pelos cgroups estudados na Aula 04.

O comportamento ao exceder o limite depende do recurso, e é a mesma distinção entre recursos compressíveis e não compressíveis da aula anterior. Exceder o limite de CPU leva apenas a *throttling*, o container é abrandado mas continua vivo. Exceder o de memória leva à terminação do container pelo OOM killer, e o kubelet regista o motivo como *OOMKilled*. A CPU mede-se em milésimos de núcleo, e `500m` equivale a meio núcleo. A memória mede-se em bytes, com sufixos como `Mi` e `Gi`.

Os *requests* e *limits* determinam também a classe de qualidade de serviço do Pod, que decide quem é sacrificado primeiro quando o nó fica sem memória. Pods sem qualquer declaração são os primeiros a ser removidos.

Para influenciar a colocação existem o *nodeSelector*, que restringe os nós por etiqueta, e a *affinity*, que exprime preferências ou obrigações de proximidade ou afastamento entre Pods. Os *taints* e *tolerations* funcionam em sentido inverso: um nó marcado com um *taint* repele os Pods que não o tolerem explicitamente, razão pela qual os nós de controlo não recebem cargas comuns.



O request é a quantidade que o container garante precisar, e é com ele que o scheduler escolhe o nó. O limit é o teto, aplicado no nó pelos cgroups da Aula 04. Entre os dois, o container pode usar mais do que pediu, se o nó o permitir.

Ao ultrapassar o limit, o comportamento depende do recurso. A CPU é compressível: o container é abrandado, o que se designa *throttling*. A memória não é: o container é terminado pelo OOM killer e o motivo fica registado como *OOMKilled*. Declarar requests realistas é o que permite ao scheduler distribuir bem a carga.

Alternativas de Orquestração

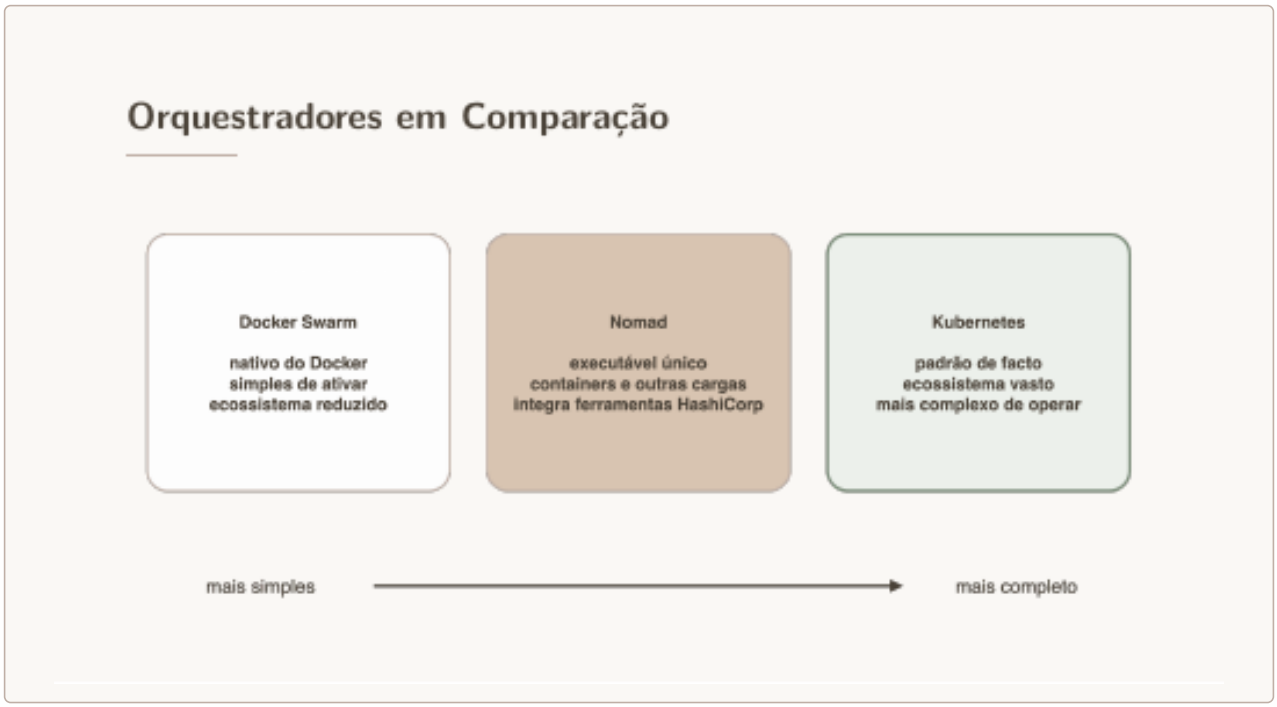
- **Docker Swarm**: solução de orquestração nativa do Docker, mais simples de configurar, mas com ecossistema e funcionalidades mais limitadas.
- **HashiCorp Nomad**: orquestrador genérico, capaz de agendar não só containers mas também outras cargas de trabalho (binários, VMs).
- Kubernetes é hoje o standard de facto, com o ecossistema mais vasto e maior adoção na indústria.
- A escolha depende da complexidade do caso de uso e da curva de aprendizagem que a equipa está disposta a assumir.

O Kubernetes não é o único orquestrador. O Docker Swarm é a solução nativa do Docker, ativada com um único comando e com uma curva de aprendizagem muito suave. Mantém o vocabulário do Docker, o que o torna acessível a quem já o conhece, mas tem um ecossistema e um conjunto de funcionalidades mais limitados e uma presença bastante inferior no mercado.

O Nomad, da HashiCorp, é um orquestrador genérico, distribuído num único executável. Agenda não só containers, mas também outras cargas, como binários diretos ou máquinas virtuais, e integra-se naturalmente com as restantes ferramentas da mesma família. É uma opção atrativa quando as cargas não são todas containers ou quando se procura menor complexidade operacional.

O Kubernetes é hoje o padrão de facto. Tem o ecossistema mais vasto, a maior comunidade e a maior oferta de serviços geridos por fornecedores de nuvem, o que reduz o risco de dependência de um único produto e de uma única equipa. Em contrapartida, é complexo de instalar e de operar.

A escolha depende, portanto, da complexidade do caso e da curva de aprendizagem que a equipa está disposta a assumir. Para um conjunto pequeno de serviços numa só equipa, uma solução mais simples pode ser a decisão correta. Para um ambiente com muitas equipas e requisitos de integração, a vantagem do ecossistema do Kubernetes pesa mais.



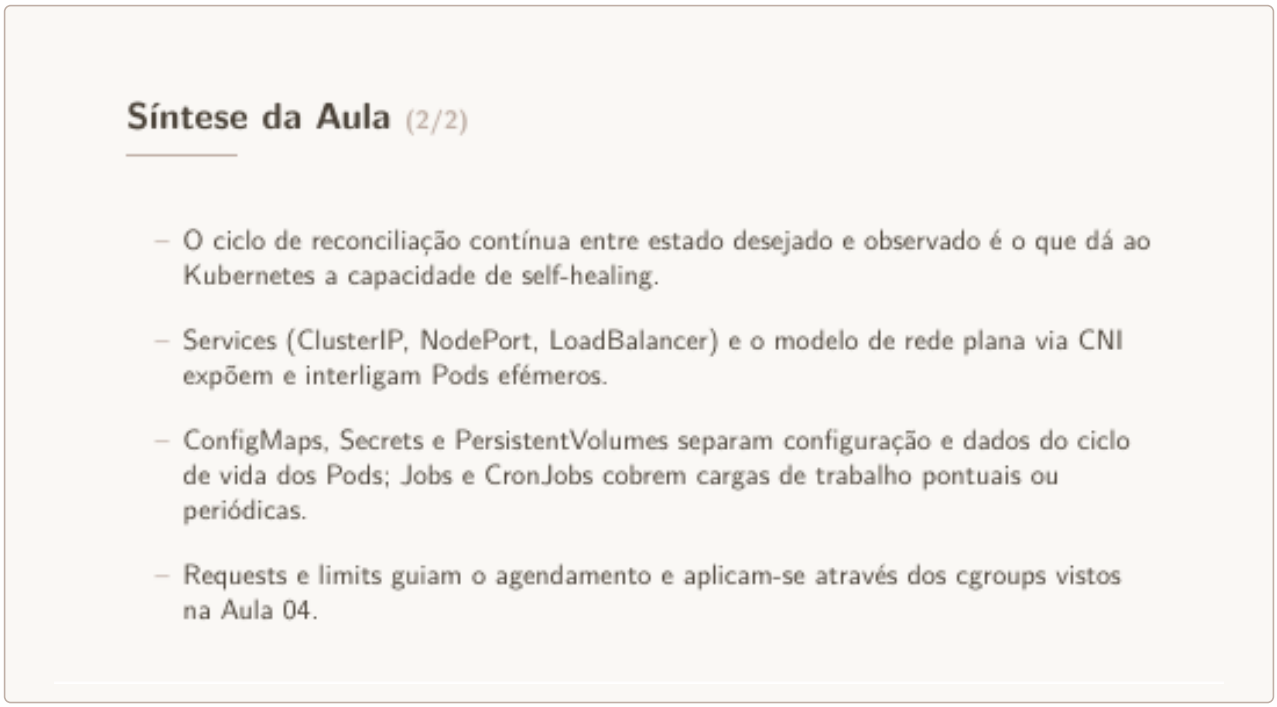
A figura ordena os três orquestradores num eixo que vai do mais simples ao mais completo. O Docker Swarm é nativo do Docker e muito fácil de ativar, mas tem menos funcionalidades e um ecossistema reduzido. O Nomad, da HashiCorp, é um executável único que agenda containers e também outras cargas, como binários e máquinas virtuais.

O Kubernetes é o padrão de facto, com o ecossistema mais vasto e a maior oferta de serviços geridos, à custa de maior complexidade. A escolha depende da dimensão do problema e da curva de aprendizagem que a equipa aceita.



A primeira metade da síntese reúne a motivação, a arquitetura e os objetos. A orquestração é necessária para gerir containers de forma fiável em vários anfitriões, decidindo onde correm e reagindo a falhas.

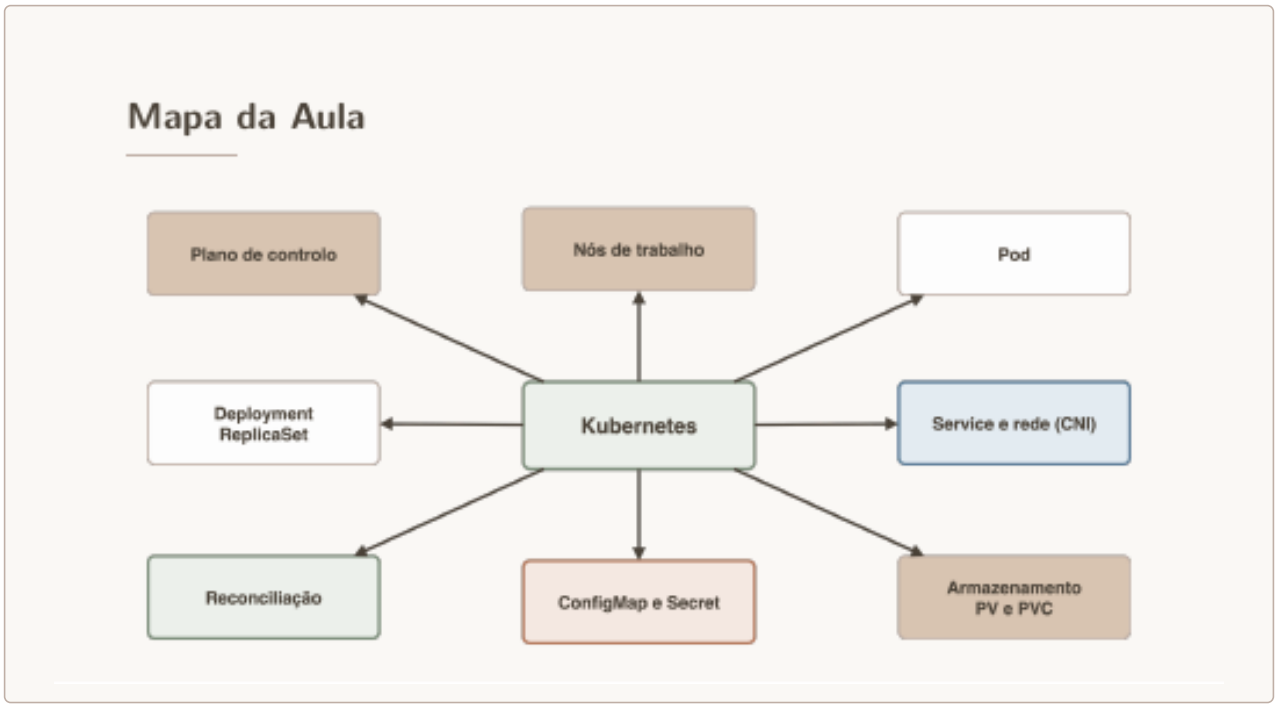
O Kubernetes separa o plano de controlo, composto pelo API server, pelo etcd, pelo scheduler e pelo controller manager, dos nós de trabalho, que têm o kubelet, o kube-proxy e o *runtime*. O Pod, o Deployment, o ReplicaSet, o Service e o Namespace são os objetos centrais para descrever e expor aplicações, numa hierarquia em que o Deployment cria ReplicaSets e estes criam Pods, e o Service os alcança por etiquetas.



A segunda metade da síntese reúne o comportamento e os recursos associados. O ciclo de reconciliação contínua entre estado desejado e estado observado é o que dá ao Kubernetes a capacidade de recuperação automática. Os Services, de tipos ClusterIP, NodePort e LoadBalancer, e o modelo de rede plana implementado por um *plugin* CNI expõem e interligam Pods efémeros.

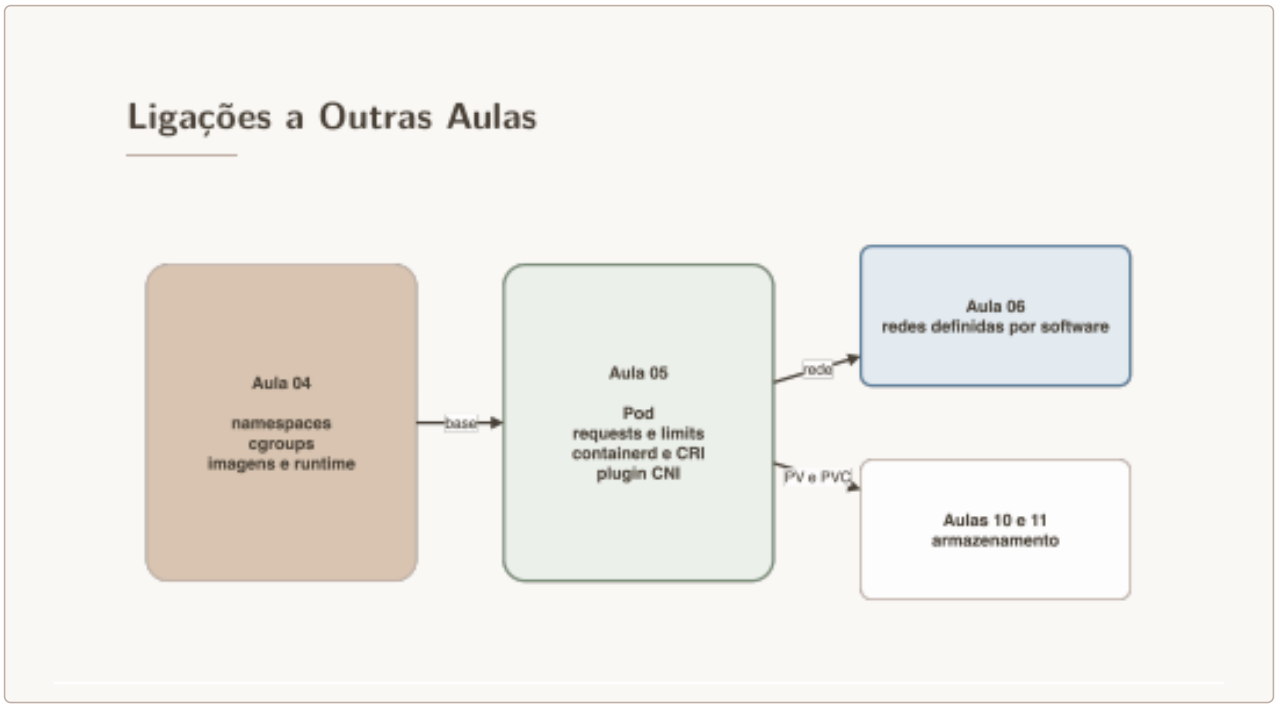
ConfigMaps, Secrets e PersistentVolumes separam a configuração e os dados do ciclo de vida dos Pods, e os Jobs e CronJobs cobrem as cargas pontuais ou periódicas. Os *requests* e *limits* guiam o agendamento e aplicam-se, no nó, através dos cgroups da Aula 04.

Como exercício de revisão, propõe-se escrever, sem consulta, o caminho completo de um pedido: do `kubectl apply` até um Pod acessível por um Service, nomeando os componentes envolvidos.



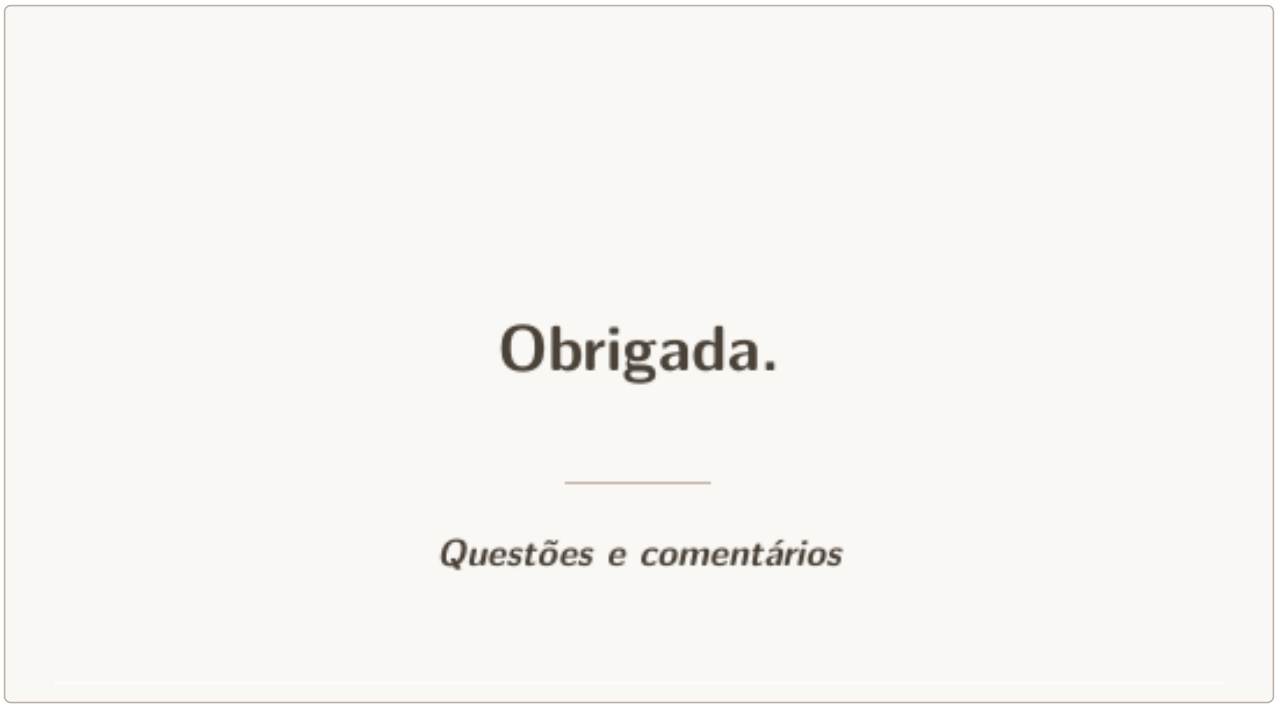
O mapa reúne os conceitos da aula em torno do Kubernetes. O plano de controlo decide e os nós de trabalho executam. O Pod é a unidade de execução, e o Deployment com o ReplicaSet garantem o estado desejado. O Service e a rede de Pods tornam as aplicações alcançáveis.

A reconciliação é o princípio que liga tudo e de que decorre a recuperação automática. O ConfigMap e o Secret separam a configuração da imagem, e o PV e o PVC separam os dados do ciclo de vida do Pod. Convém que o estudante consiga percorrer o mapa e justificar cada ligação.



A Aula 05 assenta diretamente na Aula 04. Os namespaces explicam o Pod, em particular a partilha do namespace de rede. Os cgroups explicam os limits. As imagens e o *runtime* são os mesmos, e o containerd, estudado a propósito do Docker, é o *runtime* que o Kubernetes usa.

A aula abre também caminhos. A Aula 06 retoma o modelo de rede no contexto das redes definidas por software. As Aulas 10 e 11 aprofundam o armazenamento em que o PV e o PVC assentam.



Conclui-se a exposição da Aula 05. A aula seguinte retoma o modelo de rede aqui apresentado, em que cada Pod tem um endereço próprio e a comunicação é plana, e coloca-o no quadro mais geral da gestão centralizada de redes e das redes definidas por software, onde se perguntará quem decide o encaminhamento e como essa decisão se separa do equipamento que a executa.